

Chapter 2:
Học tập có giám sát

Ph.D. Nguyen Thi Khanh Tien

tienntk@ut.edu.vn

Giới thiệu về học tập có giám sát

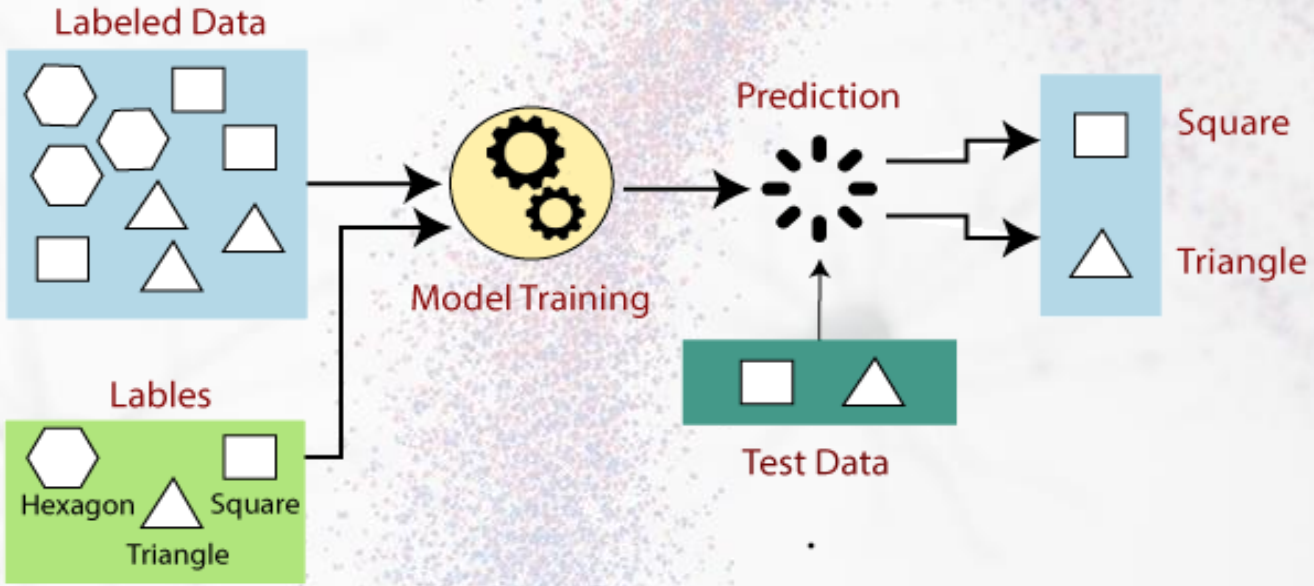
Học có giám sát là một kỹ thuật học máy trong đó mô hình được đào tạo dựa trên dữ liệu được dán nhãn. Điều này có nghĩa là mô hình được cung cấp cả dữ liệu đầu vào và đầu ra chính xác tương ứng. Bằng cách phân tích các ví dụ này, mô hình học cách dự đoán kết quả đầu ra cho dữ liệu mới, chưa được nhìn thấy.

Các thuật toán học có giám sát nhằm mục đích xác định mô hình hoặc mối quan hệ giữa các biến đầu vào và đầu ra, ánh xạ hiệu quả dữ liệu đầu vào (x) với dữ liệu đầu ra (y). Chức năng ánh xạ đã học này sau đó có thể được sử dụng để đưa ra dự đoán về dữ liệu mới.

Các ứng dụng trong thế giới thực của học tập có giám sát bao gồm các nhiệm vụ như đánh giá rủi ro, phân loại hình ảnh, phát hiện gian lận, lọc thư rác, v.v.

Ví dụ:

- Email ⟶ Thư rác hay không phải thư rác
- Hình ảnh ⟶ Mèo hoặc Chó
- Tính năng nhà cái⟶Dự đoán giá

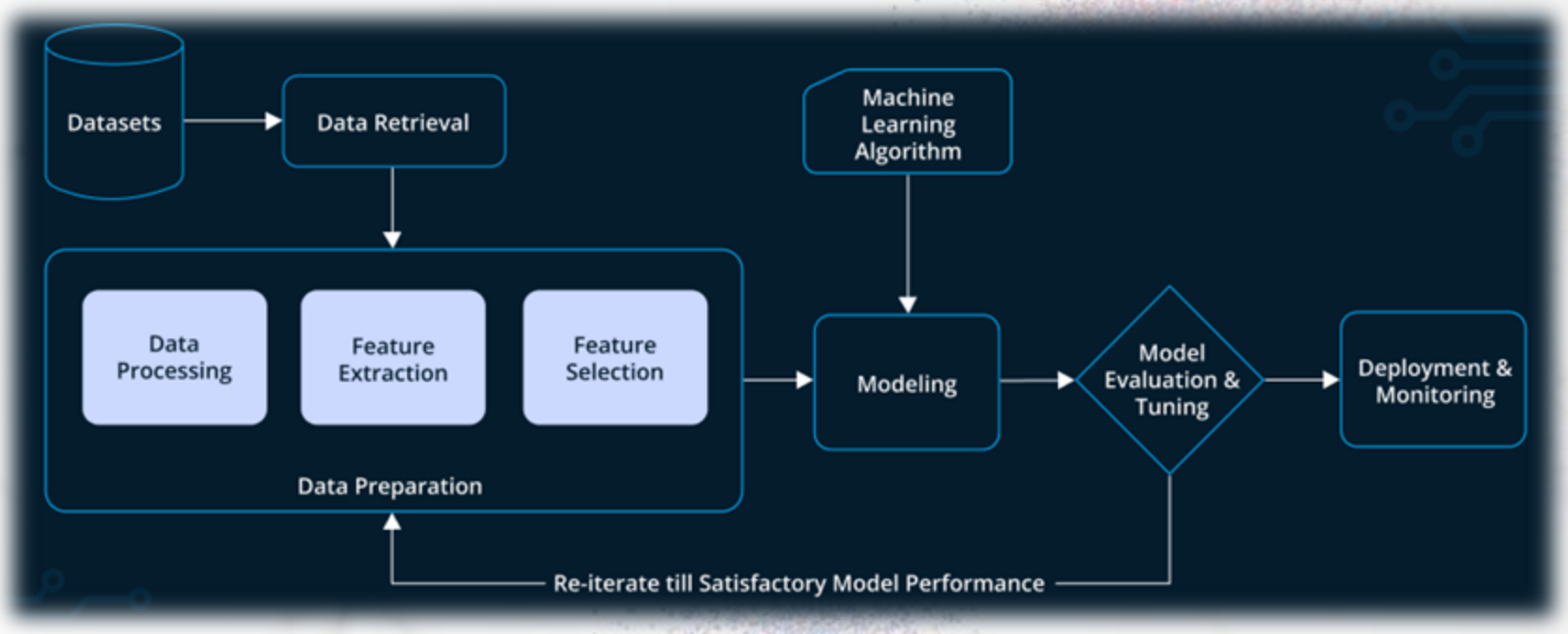


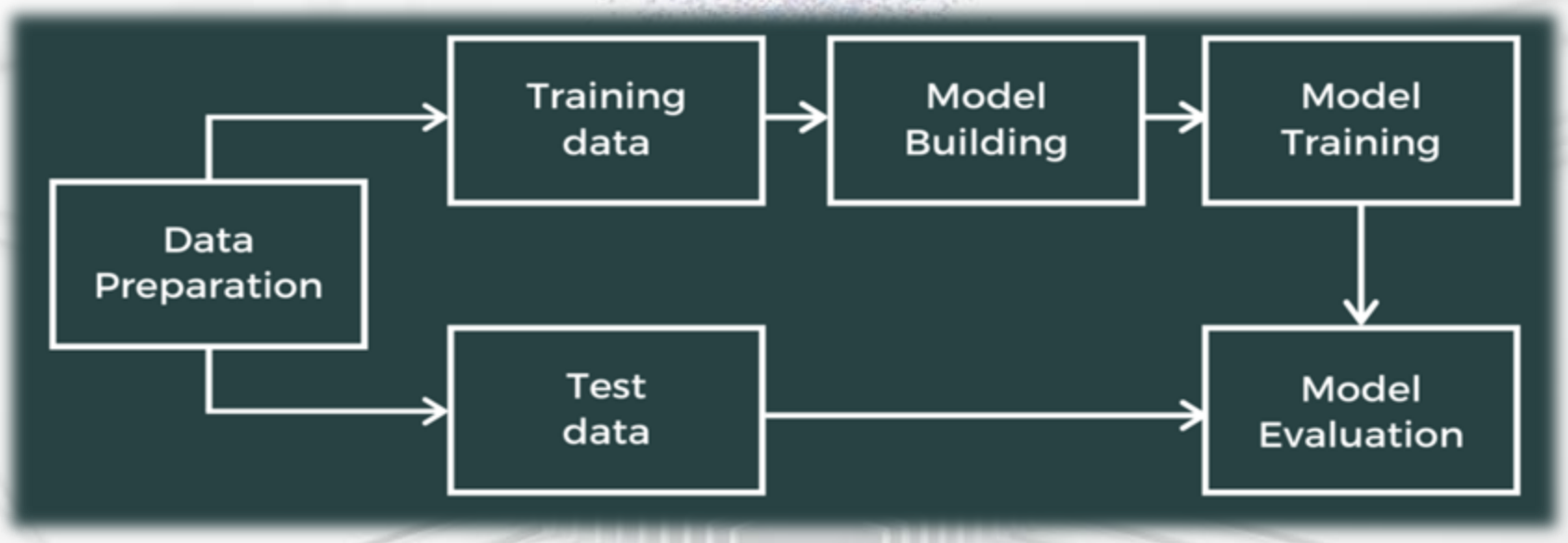
Cách thức hoạt động:

1. Dữ liệu đầu vào (X): Tính năng hoặc quan sát

2. Nhãn đầu ra (Y): Các câu trả lời đúng (mục tiêu) **3. Đào tạo mô hình:** Thuật toán học ánh xạ từ X ⟶ Y **4. Dự đoán:** Mô hình sử dụng ánh xạ này để dự đoán Y cho X **mới 5. Đánh giá:** Hiệu suất được đo bằng các số liệu như độ chính xác, độ chính xác, v.v.

Steps Involved in Supervised Learning





Các bước liên quan đến học tập có giám sát

Thu thập và chuẩn bị dữ liệu:

- **Thu thập dữ liệu:** Thu thập tập dữ liệu có liên quan đến vấn đề bạn muốn giải quyết.
- **Làm sạch và tiền xử lý:** Loại bỏ nhiễu, xử lý các giá trị bị thiếu và chuẩn hóa hoặc chuẩn hóa dữ liệu để đảm bảo tính nhất quán.
- **Ghi nhãn:** Gán nhãn hoặc đầu ra chính xác cho từng điểm dữ liệu.

Chọn mẫu:

- **Chọn thuật toán:** Xem xét các yếu tố như bản chất của dữ liệu của bạn (ví dụ: số, phân loại), đầu ra mong muốn (ví dụ: phân loại, hồi quy) và tài nguyên tính toán.

- **Các thuật toán phổ biến:** hồi quy tuyến tính, hồi quy logistic, cây quyết định, rừng ngẫu nhiên, máy vectơ hỗ trợ (SVM), mạng nơ-ron,..

Chia dữ liệu:

- **Tập huấn luyện:** Sử dụng phần dữ liệu này để huấn luyện mô hình.
- **Bộ xác thực:** Đánh giá hiệu suất của mô hình trong quá trình đào tạo và tinh chỉnh siêu tham số.
- **Bộ kiểm tra:** Đánh giá hiệu suất cuối cùng của mô hình trên dữ liệu chưa xem.

Đào tạo:

- **Quy trình lặp lại:** Mô hình học từ dữ liệu huấn luyện bằng cách điều chỉnh các tham số của nó để giảm thiểu sự khác biệt giữa kết quả đầu ra được dự đoán và nhãn thực tế.
- **Tối ưu hóa:** Sử dụng các kỹ thuật như giảm độ dốc để tìm các giá trị tham số tối ưu.

Đánh giá:

- **Số liệu:** Sử dụng số liệu thích hợp để đo lường hiệu suất của mô hình trên các tập hợp kiểm tra và xác thực.
- **Các số liệu phổ biến:** Các ví dụ bao gồm độ chính xác, độ chính xác, khả năng thu hồi, điểm F1, sai số bình phương trung bình (MSE) và sai số bình phương trung bình gốc (RMSE).

Triển khai:

- **Tích hợp:** Sau khi hài lòng với hiệu suất của mô hình, hãy triển khai nó vào môi trường sản xuất.
- **Giám sát:** Liên tục theo dõi hiệu suất của mô hình và đào tạo lại mô hình nếu cần thiết để thích ứng với dữ liệu hoặc điều kiện thay đổi.

Ưu điểm và nhược điểm của việc học có giám sát



ADVANTAGES

- **High Accuracy:** Produces reliable and accurate predictions with sufficient labeled data
- **Clear Objective:** Learns a direct mapping from input to output
- **Wide Applications:** Used in classification, regression, fraud detection, medical diagnosis, etc.

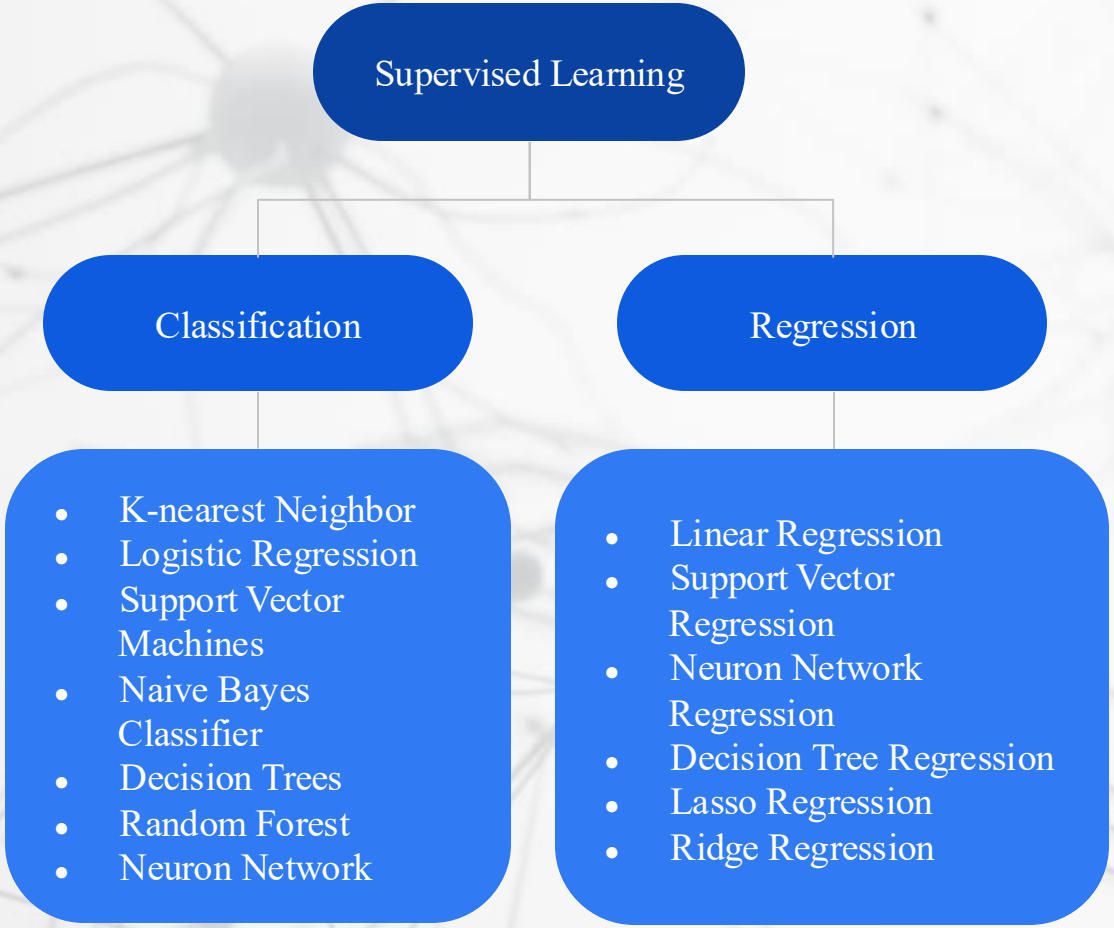


DISADVANTAGES

- **Needs Labeled Data:** Labeling large datasets is time-cons-uming and ad costly
- **Limited to Known Classes:** Cannot handle unseen classes or patterns well
- **Overfitting Risk:** May memorize training data and perform poorly on new data
- **Scalability Issues:** Training time can increase with complex or large datasets

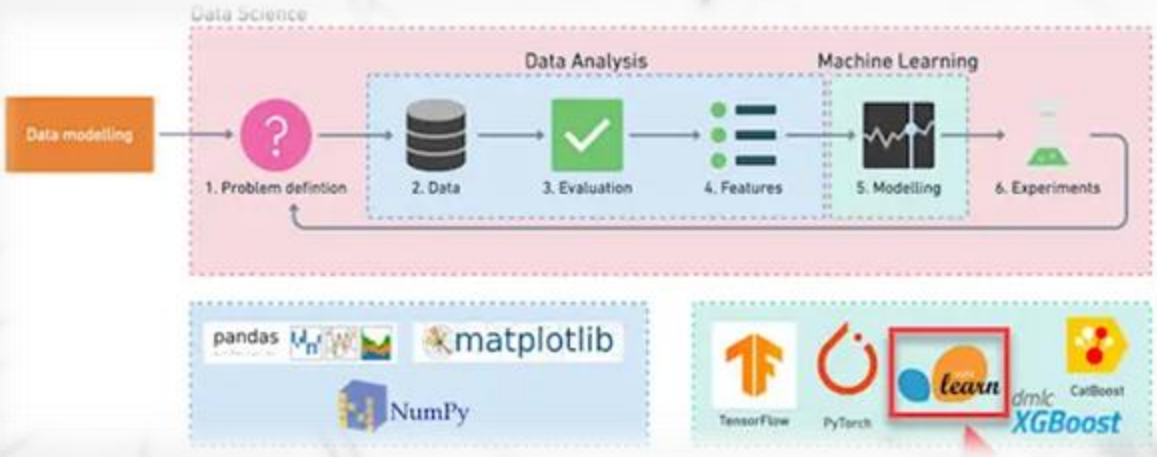
Các loại hình học tập có giám sát

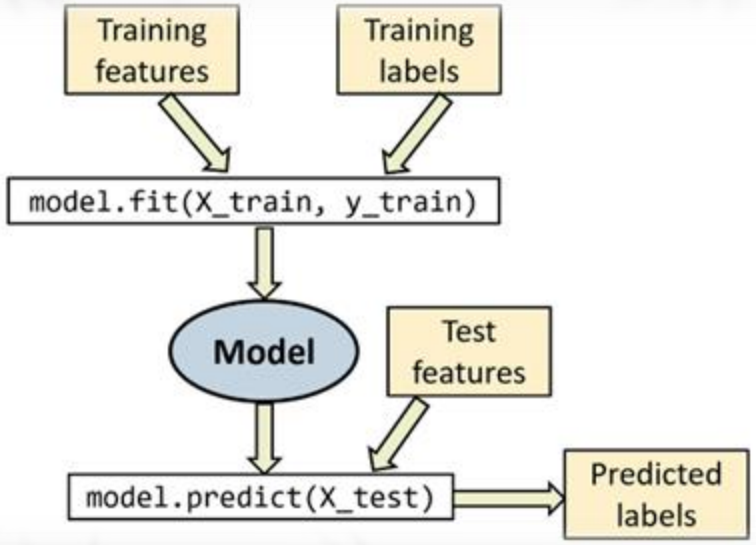
Dựa trên các tập dữ liệu đã cho, vấn đề SL được phân thành hai loại, ***Phân loại*** và ***Hồi quy.***

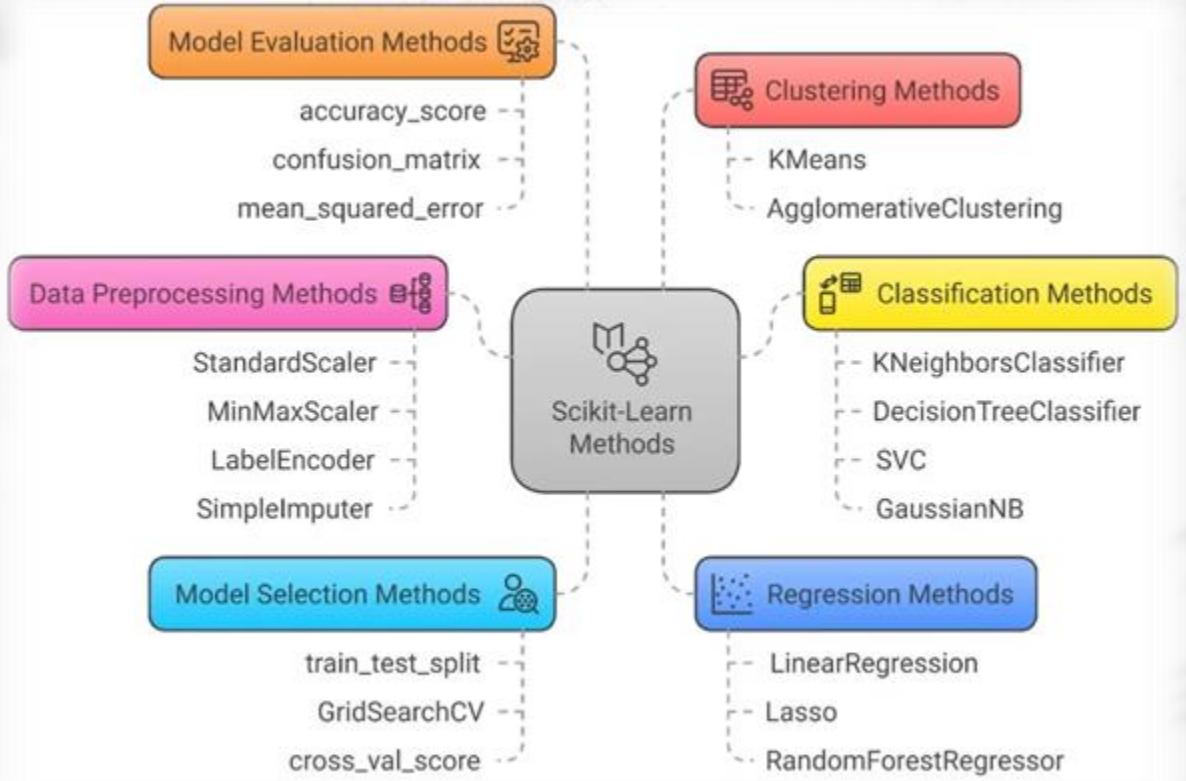


	Phân loại	**Hồi quy**
Mục tiêu	Để dự đoán kết quả phân loại	Để dự đoán số liên tục giá trị
Đầu ra loại	Phân loại (ví dụ: có/không, lớp nhãn)	Liên tục (ví dụ: số)
Các chỉ số đánh giá	Độ chính xác, Độ chính xác, Thu hồi, Điểm F1,...	Nghĩa là Lỗi bình phương (MSE), Trung bình Lỗi tuyệt đối (MAE), Root Lỗi bình phương trung bình (RMSE), bình phương R
Ví dụ	Thư rác hình ảnh phát hiện sự công nhận Dự đoán rời bỏ khách hàng	Dự đoán giá nhà Cổ phiếu giá cảdự đoán dự báo bán hàng

Phương pháp học Scikit cho ML

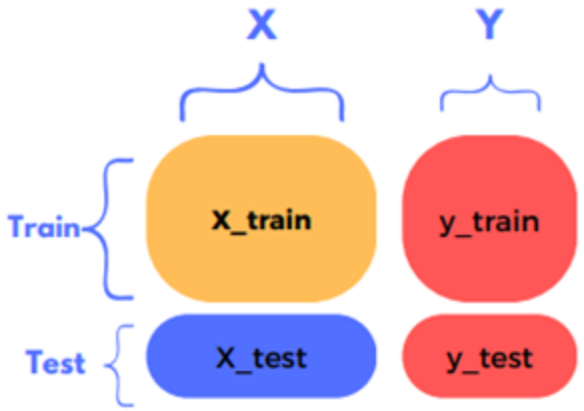






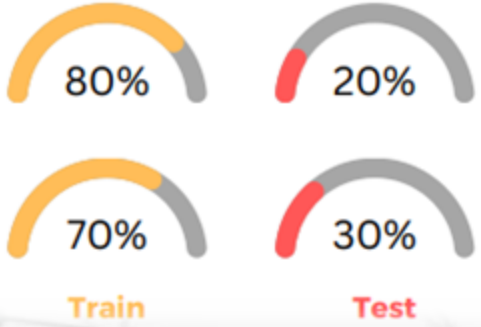
Scikit-learn Methods for Supervised Learning

Train-Test Split



- **Method of dividing the data into two part: Training and Testing**
- **Training Data : Model Training**
- **Testing Data : Model Evaluation**

Standard Ratio




```
from sklearn.model_selection import
train_test_split
X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3)
```

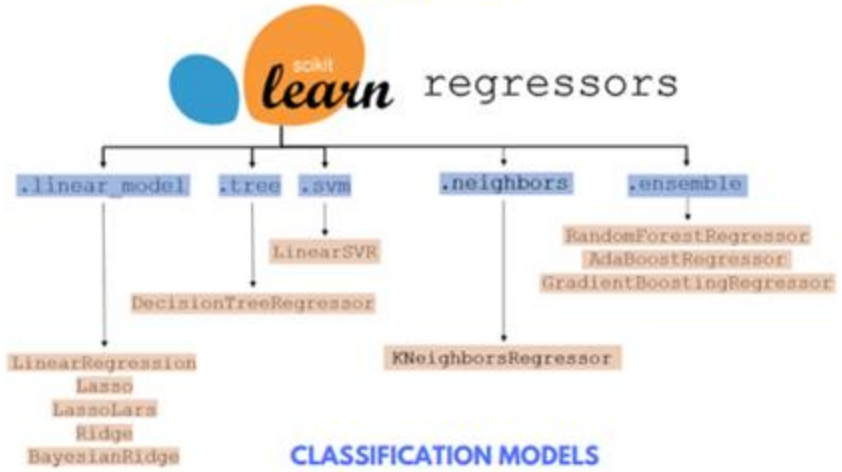
```
#Import
from sklearn.branch import
model_name

# Create a Instance
model = model_name()

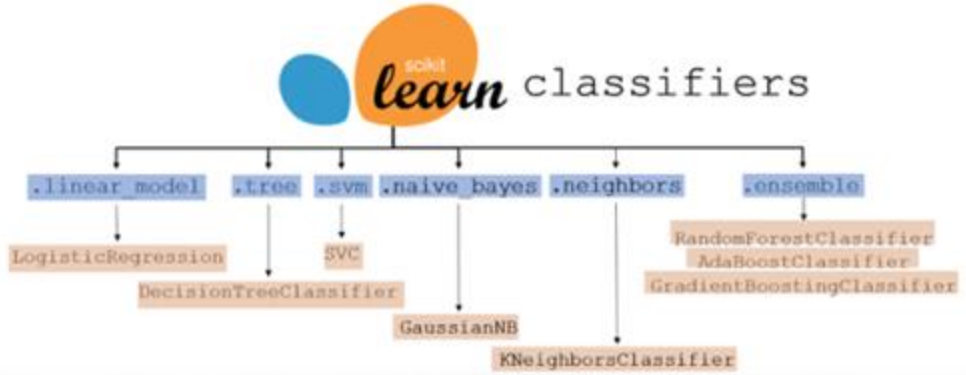
# fit model
model.fit(X_train, y_train)
```

Supervised Learning Models

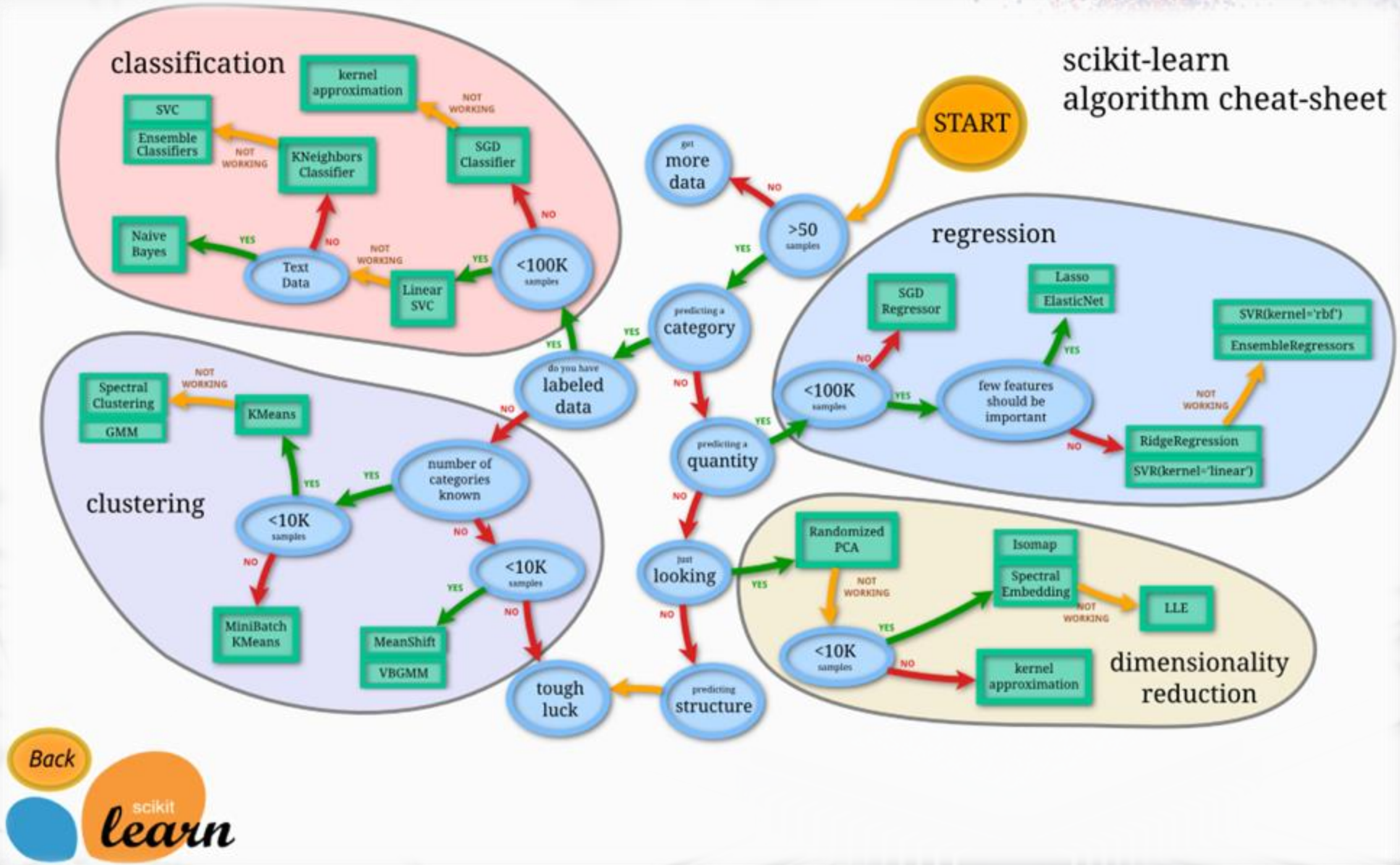
REGRESSION MODELS



CLASSIFICATION MODELS



Thuật toán Scikit-learn



KDnuggets | Visit KDnuggets.com for more cheatsheets and additional learning resources.

Scikit-learn CheatSheet



Scikit-learn is an open-source Python library for all kinds of predictive data analysis. You can perform classification, regression, clustering, dimensionality reduction, model tuning, and data preprocessing tasks.

Loading the Data

Classification

```
from sklearn import datasets
X, y = datasets.load_wine(return_X_y=True)
```

Regression

```
diabetes = datasets.load_diabetes()
X, y = diabetes.data, diabetes.target
```

Training And Test Data

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0
)
```

Preprocessing the Data

Standardization

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_X_train = scaler.fit_transform(X_train)
scaled_X_test = scaler.transform(X_test)
```

Normalization

```
from sklearn.preprocessing import Normalizer
norm = Normalizer()
norm_X_train = norm.fit_transform(X_train)
norm_X_test = norm.transform(X_test)
```

Binarization

```
from sklearn.preprocessing import Binarizer
binary = Binarizer(threshold=0.0)
binary_X = binary.fit_transform(X)
```

Encoding Categorical Features

```
from sklearn.preprocessing import LabelEncoder
lab_enc = LabelEncoder()
y = lab_enc.fit_transform(y)
```

Imputer

```
from sklearn.impute import SimpleImputer
imp_mean = SimpleImputer(missing_values=0,
strategy='mean')
imp_mean.fit_transform(X_train)
```

Supervised Learning Model

Linear Regression

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

Support Vector Machines

```
from sklearn.svm import SVC
svm_svc = SVC(kernel='linear')
```

Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
```

Unsupervised Learning Model

Principal Component Analysis

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
```

K Means

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=5, random_state=0)
```

Model Fitting

Supervised Learning

```
lr.fit(X_train, y_train)
svm_svc.fit(X_train, y_train)
```

Unsupervised Learning

```
model = pca.fit_transform(X_train)
kmeans.fit(X_train)
```

Prediction

Supervised Learning

```
y_pred = lr.predict_proba(X_test)
y_pred = svm_svc.predict(X_test)
```

Unsupervised Learning

```
y_pred = kmeans.predict(X_test)
```

Evaluation

Accuracy Score

```
lr.score(X_test, y_test)

from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

Classification Report

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Mean Squared Error

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_test, y_pred)
```

R2 Score

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

Adjusted Rand Index

```
from sklearn.metrics import adjusted_rand_score
adjusted_rand_score(y_test, y_pred)
```

Cross-Validation

```
from sklearn.model_selection import cross_val_score
cross_val_score( lr, X, y, cv=5, scoring='f1_macro')
```

Model Tuning

```
from sklearn.model_selection import GridSearchCV
parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
model = GridSearchCV(svm_svc, parameters)
model.fit(X_train, y_train)
print(model.best_score_)
print(model.best_estimator_)
```

Subscribe to KDnuggets News

Ví dụ: Phân loại hoa diên vĩ bằng Scikit -learn

Vấn đề: Dự đoán loài hoa Iris dựa trên chiều dài đài hoa, chiều rộng đài hoa, chiều dài cánh hoa và chiều rộng cánh hoa. Đây là một vấn đề phân loại nhiều lớp.

Bước 1: Tải dữ liệu

Scikit-learn đi kèm với một số bộ dữ liệu đồ chơi, bao gồm cả bộ dữ liệu Iris, rất phù hợp để trình diễn.

- *load_iris()*: Tìm nạp tập dữ liệu Iris.
- *iris.data*: Chứa các phép đo tính năng (chiều dài đài hoa, chiều rộng đài hoa, chiều dài cánh hoa, chiều rộng cánh hoa).
- *iris.feature_names*: Tên của các tính năng.
- *iris.target*: Chứa nhãn loài (0, 1, 2 đại diện cho 'setosa', 'versicolor', 'virginica').
- *iris.target_names*: Tên thật của các lớp mục tiêu.
- Chuyển đổi *X* thành Pandas DataFrame và *y* thành *Pandas Series* để dễ đọc và thao tác tốt hơn, mặc dù mảng *NumPy* hoạt động hoàn toàn tốt với Scikit-learn.

```python
# *** Step 1: Load Data ***
# Import necessary libraries
from sklearn.datasets import load_iris
import pandas as pd

# Load the Iris dataset
iris = load_iris()

# The data is stored in .data attribute
X = pd.DataFrame(iris.data, columns=iris.feature_names)

# The target variable is stored in .target attribute
y = pd.Series(iris.target)

# Display the first 5 rows of the features
print("Features (X):")
print(X.head())

# Display the first 5 target values and their corresponding names
print("\nTarget (y) and Target Names:")
print(y.head())
print("Target names:", iris.target_names)
```

Ví dụ: Phân loại hoa diên vĩ bằng Scikit -learn

Bước 2: Tách dữ liệu

- *train_test_split()*: Hàm này chia dữ liệu một cách ngẫu nhiên.
- *test_size=0,30*: 30% dữ liệu sẽ được phân bổ cho tập thử nghiệm và 70% còn lại cho tập huấn luyện.
- *random_state=42*: Điều này đảm bảo rằng sự phân tách giống nhau mỗi khi bạn chạy mã, điều này rất quan trọng đối với khả năng tái tạo.

Bước 3: Tiền xử lý dữ liệu

Đối với tập dữ liệu cụ thể này và Hồi quy logistic, việc chia tỷ lệ có thể không có tác động đáng kể vì các tính năng đã ở quy mô tương tự. Tuy nhiên, đó là một cách thực hành tốt, đặc biệt đối với các thuật toán nhạy cảm với thang đo tính năng (như SVM, KNN, mạng thần kinh).

- *StandardScaler()*: Chuẩn hóa các tính năng bằng cách loại bỏ giá trị trung bình và chia tỷ lệ thành phương sai đơn vị. Điều này đảm bảo tất cả các tính năng đều đóng góp như nhau cho mô hình.
- *fit_transform(X_train)*: Tính giá trị trung bình và độ lệch chuẩn từ *X_train* và sau đó áp dụng phép biến đổi.
- *transform(X_test)*: Áp dụng *giá trị trung bình và độ lệch chuẩn tương tự* đã học được từ *X_train* đến *X_test*. Bạn **không bao giờ** lắp bộ chia tỷ lệ vào dữ liệu thử nghiệm.

```python
# *** Step 2: Split Data ***
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
# test_size=0.30 means 30% of the data will be used for testing
# random_state for reproducibility (same split every time)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.30,
                                                    random_state=42)

print("\nShape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

```python
# *** Step 3: Preprocess Data ***
# Standardization is a common preprocessing step for many machine learning algorithms
# It scales the features to have a mean of 0 and a standard deviation of 1
from sklearn.preprocessing import StandardScaler

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform both training and testing data
# IMPORTANT: Fit only on training data to avoid data leakage from the test set
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFirst 5 rows of X_train (original):")
print(X_train.head())

print("\nFirst 5 rows of X_train_scaled (after standardization):")
print(X_train_scaled[:5])
```

Ví dụ: Phân loại hoa diên vĩ bằng Scikit -learn

Bước 4: Chọn mẫu

Đối với một vấn đề phân loại nhiều lớp như Iris, Hồi quy logistic là điểm khởi đầu tốt do tính đơn giản và dễ hiểu của nó.

- *LogisticRegression()*: Khởi tạo trình phân loại hồi quy logistic.

- *bộ giải='libTuyến tính'*: Chỉ định thuật toán sử dụng để tối ưu hóa. Các bộ giải khác nhau phù hợp với các kích thước và loại tập dữ liệu khác nhau.
- *random_state*: Đảm bảo các quy trình ngẫu nhiên bên trong của mô hình (nếu có) có thể lặp lại được.

```
# *** Step 4: Choose a Model ***
# For this example, we will use Logistic Regression as a classification model
# Logistic Regression is suitable for binary and multiclass classification problems
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# solver='liblinear' is a good choice for smaller datasets
# random_state for reproducibility of the model's internal randomness
model = LogisticRegression(solver='liblinear', random_state=42)

print("\nChosen Model:", model)
```

Bước 5: Huấn luyện mô hình

- *model.fit(X_train_scaled, y_train)*: Đây là nơi điều kỳ diệu xảy ra! Mô hình tìm hiểu các mẫu và mối quan hệ giữa các tính năng (*X_train_scaled*) và nhãn mục tiêu (*y_train*).

```
# *** Step 5: Train the Model ***
# Fit the model on the training data
model.fit(X_train_scaled, y_train)

print("\nModel trained successfully!")
```

Bước 6: Đưa ra dự đoán

- *model.predict(X_test_scaled)*: Mô hình được đào tạo lấy các tính năng kiểm tra tỷ lệ làm đầu vào và xuất ra lớp dự đoán của nó cho mỗi mẫu

```
# *** Step 6: Make Predictions ***
# Use the trained model to make predictions on the test set
y_pred = model.predict(X_test_scaled)
# print("\nPredictions on the test set:")
# print(y_pred)
print("\nFirst 10 actual labels (y_test):", y_test.values[:10])
print("First 10 predicted labels (y_pred):", y_pred[:10])
```

✓ 0.0s

Ví dụ: Phân loại hoa diên vĩ bằng Scikit -learn

Bước 7: Đánh giá mô hình

Đánh giá mô hình của bạn hoạt động tốt như thế nào trên tập thử nghiệm bằng nhiều số liệu khác nhau. Để phân loại, các số liệu phổ biến bao gồm độ chính xác, độ chính xác, khả năng thu hồi và điểm F1, thường được tóm tắt trong báo cáo phân loại và ma trận nhầm lẫn.

- *accuracy_score()*: Tính tỷ lệ mẫu được phân loại chính xác.
- *classification_report()*: Cung cấp bản tóm tắt chi tiết về độ chính xác, khả năng thu hồi và điểm F1 cho từng lớp cũng như hỗ trợ tổng thể.
- *Độ chính xác*: Trong số tất cả các mẫu được dự đoán là một lớp nhất định, có bao nhiêu mẫu thực sự thuộc lớp đó?
- *Nhớ lại*: Trong số tất cả các mẫu thực sự thuộc về một lớp nhất định, có bao nhiêu mẫu được dự đoán chính xác là lớp đó?
- *F1-Điểm*: Giá trị trung bình hài hòa của độ chính xác và khả năng thu hồi, cung cấp thước đo cân bằng.
- *confusion_matrix()*: Bảng hiển thị số lượng dự đoán đúng và sai cho mỗi lớp. Thật tuyệt vời khi biết mô hình của bạn mắc lỗi ở đâu.

```python
# *** Step 7: Evaluate the Model ***
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.4f}")

# Generate a classification report (precision, recall, f1-score per class)
class_report = classification_report(y_test, y_pred, target_names=iris.target_names)
print("\nClassification Report:\n", class_report)

# Generate a confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", conf_matrix)

# Interpretation of Confusion Matrix:
# Rows represent true labels, columns represent predicted labels.
# conf_matrix[i, j] is the number of observations known to be in group i
# and predicted to be in group j.
# For example, in a 3x3 matrix:
# [[TN, FP, FP],
#  [FN, TP, FN],
#  [FN, FN, TN]]
# (This interpretation is more for binary. For multi-class, it's just true vs predicted counts)
print("\n--- Confusion Matrix Interpretation ---")
print("Rows: True Labels")
print("Columns: Predicted Labels")
print(pd.DataFrame(conf_matrix, index=iris.target_names, columns=iris.target_names))
```

Ví dụ: Phân loại hoa diên vĩ bằng Scikit -learn

Bước 7: Đánh giá mô hình

Đánh giá mức độ hiệu quả của mô hình của bạn trên tập thử nghiệm bằng nhiều số liệu khác nhau. Để phân loại, các số liệu phổ biến bao gồm độ chính xác, độ chính xác, khả năng thu hồi và điểm F1, thường được tóm tắt trong báo cáo phân loại và ma trận nhầm lẫn.

- *accuracy_score()*: Tính tỷ lệ mẫu được phân loại chính xác.

- *classification_report()*: Cung cấp bản tóm tắt chi tiết về độ chính xác, khả năng thu hồi và điểm F1 cho từng lớp cũng như hỗ trợ tổng thể.
- **Độ chính xác:** Trong số tất cả các mẫu được dự đoán là một lớp nhất định, có bao nhiêu mẫu thực tế thuộc lớp đó?
- **Nhớ lại:** Trong số tất cả các mẫu thực sự thuộc về một lớp nhất định, có bao nhiêu mẫu được dự đoán chính xác là lớp đó?
- **F1-Điểm:** Giá trị trung bình hài hòa của độ chính xác và khả năng thu hồi, mang lại thước đo cân bằng.
- *confusion_matrix()*: Bảng hiển thị số lượng dự đoán đúng và sai cho mỗi lớp. Thật tuyệt vời khi biết mô hình của bạn mắc lỗi ở đâu.

```python
# *** Step 7: Evaluate the Model ***
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.4f}")

# Generate a classification report (precision, recall, f1-score per class)
class_report = classification_report(y_test, y_pred, target_names=iris.target_names)
print("\nClassification Report:\n", class_report)

# Generate a confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", conf_matrix)

# Interpretation of Confusion Matrix:
# Rows represent true labels, columns represent predicted labels.
# conf_matrix[i, j] is the number of observations known to be in group i
# and predicted to be in group j.
# For example, in a 3x3 matrix:
# [[TN, FP, FP],
#  [FN, TP, FN],
#  [FN, FN, TN]]
# (This interpretation is more for binary. For multi-class, it's just true vs predicted counts)
print("\n--- Confusion Matrix Interpretation ---")
print("Rows: True Labels")
print("Columns: Predicted Labels")
print(pd.DataFrame(conf_matrix, index=iris.target_names, columns=iris.target_names))
```

Ví dụ: Phân loại hoa diên vĩ bằng Scikit -learn

Bước 8: Điều chỉnh siêu tham số

Điều chỉnh siêu tham số liên quan đến việc tìm bộ siêu tham số tốt nhất cho mô hình của bạn để tối ưu hóa hiệu suất của mô hình. *GridSearchCV* là một phương pháp phổ biến cho việc này.

- *GridSearchCV*: Tìm kiếm một cách có hệ thống thông qua tập hợp các giá trị siêu tham số được xác định trước, đánh giá từng kết hợp bằng xác thực chéo và chọn kết hợp mang lại hiệu suất tốt nhất.

- *param_grid*: Một từ điển trong đó khóa là tên siêu tham số và giá trị là danh sách các giá trị cần thử.
- *cv=5*: Thực hiện xác thực chéo 5 lần, nghĩa là dữ liệu huấn luyện được chia thành 5 phần và mô hình được huấn luyện 5 lần, mỗi lần sử dụng một phần khác nhau làm bộ xác thực.
- *cho điểm='độ chính xác'*: Số liệu được sử dụng để đánh giá mô hình trong quá trình xác thực chéo.
- *grid_search.best_params_*: Sự kết hợp siêu tham số mang lại hiệu suất tốt nhất.
- *grid_search.best_score_*: Điểm xác thực chéo đạt được với siêu tham số tốt nhất.
- *grid_search.best_estimator_*: Mô hình được đào tạo thực tế với các siêu tham số tốt nhất.

```python
# *** Step 8: Tune Hyperparameters ***
# Hyperparameter tuning can be done using techniques like Grid Search or Random Search.
from sklearn.model_selection import GridSearchCV

# Define the parameter grid to search
param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100],  # Inverse of regularization strength
    'solver': ['liblinear', 'lbfgs']     # Algorithms to use in the optimization problem
}

# Initialize GridSearchCV
# estimator: The model to tune
# param_grid: The dictionary of hyperparameters to try
# cv: Number of cross-validation folds
# scoring: The metric to optimize (e.g., 'accuracy' for classification)
grid_search = GridSearchCV(LogisticRegression(random_state=42),
                           param_grid, cv=5, scoring='accuracy')

# Fit GridSearchCV to the scaled training data
grid_search.fit(X_train_scaled, y_train)

print("\nBest Hyperparameters found:", grid_search.best_params_)
print("Best Cross-validation Accuracy:", grid_search.best_score_)

# Get the best model
best_model = grid_search.best_estimator_

# Evaluate the best model on the test set
y_pred_tuned = best_model.predict(X_test_scaled)
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
print(f"Accuracy with Tuned Model on Test Set: {accuracy_tuned:.4f}")

# You can also get a classification report for the tuned model
print("\nClassification Report with Tuned Model:\n",
      classification_report(y_test, y_pred_tuned, target_names=iris.target_names))
```

Học máy để phân loại

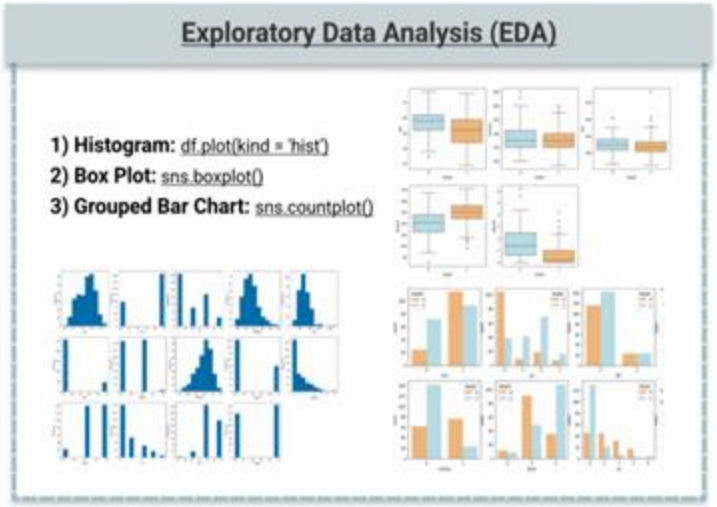
Phân loại là một kỹ thuật học có giám sát dùng để dự đoán danh mục hoặc lớp điểm dữ liệu mới dựa trên một tập hợp các ví dụ đào tạo được gắn nhãn. Trong phân loại, một mô hình học các mẫu từ dữ liệu huấn luyện để phân loại các quan sát chưa nhìn thấy thành các lớp hoặc nhóm được xác định trước.

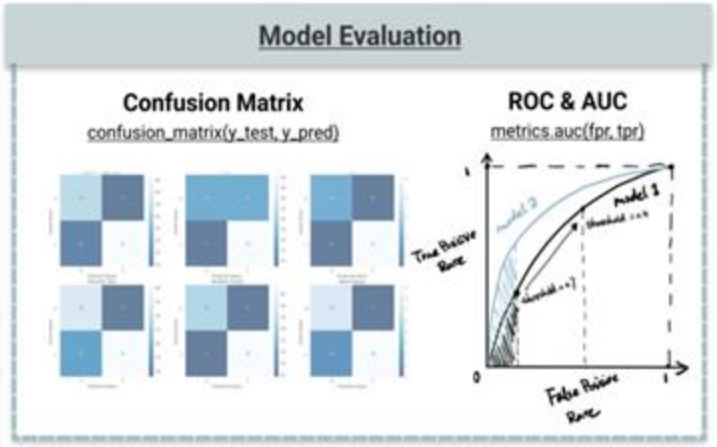
Hai loại Phân loại:

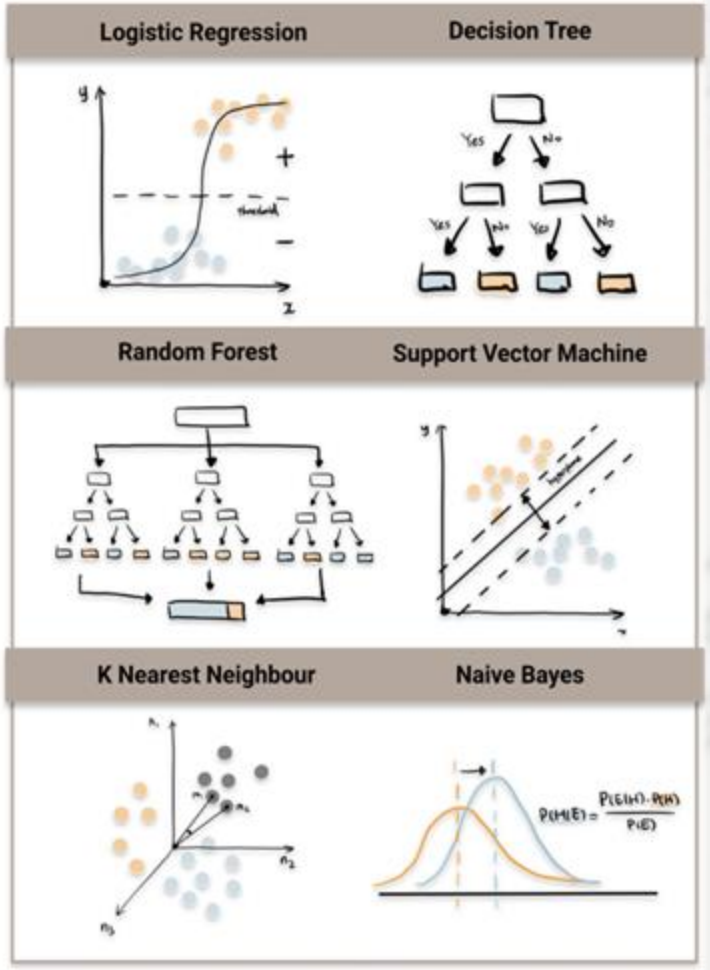
- **Phân loại nhị phân:** Dự đoán kết quả chỉ với hai loại có thể xảy ra (ví dụ: spam hay không spam, nam hay nữ, có hoặc không)
- **Phân loại nhiều lớp:**dự đoán các kết quả có nhiều hơn hai lớp có thể xảy ra (ví dụ: phân loại các loại cây trồng, phân loại các loại âm nhạc)

Thuật toán phân loại được sử dụng rộng rãi trong nhiều lĩnh vực khác nhau, bao gồm:

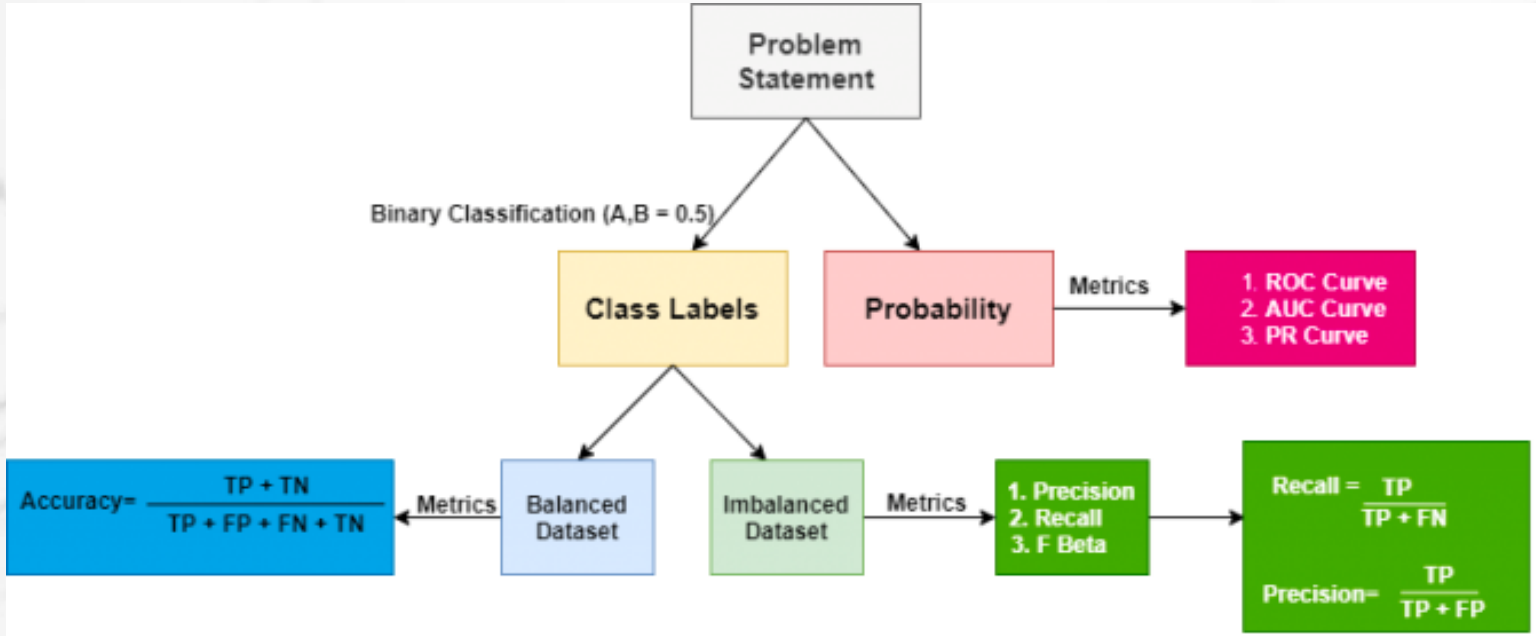
- Chăm sóc sức khỏe: Chẩn đoán bệnh
- Tài chính: Phát hiện gian lận
- Tiếp thị: Phân khúc khách hàng
- Xử lý hình ảnh: Nhận dạng đối tượng

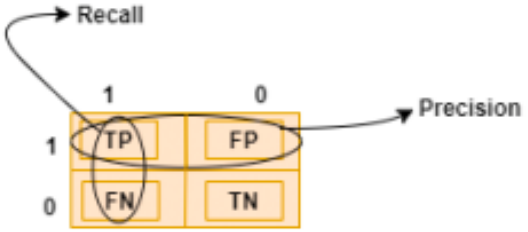




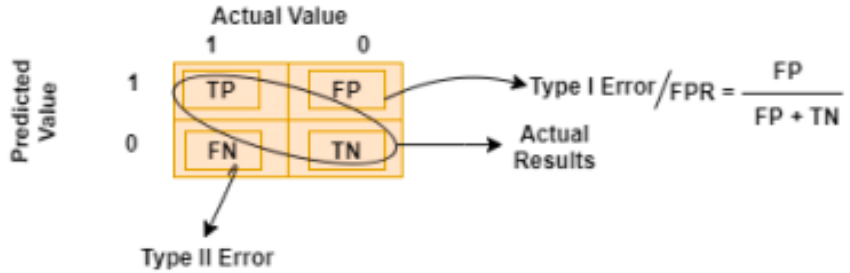


Classification Metrics in Machine Learning





Confusion Matrix



Always have to focus on reducing the Type I Error and Type II Error

Recall (TPR) : Out of the total positive actual value, how many values did we correctly predicted positively. Whenever the FN is important wrt the problem statement, use Recall.

Precision : Out of the total predicted positive results, how many results were actual positive. Whenever the FP is important wrt the problem statement, then use Precision

In some of the problem statements FP and FN both are equally important in an imbalanced dataset.In such cases we have to consider both **Recall** and **Precision.** In such cases we have to use **F-beta score**

$$\text{F-Beta} = \frac{(1+B^2)\ \text{Precision X Recall}}{B^2 \times (\text{Precision} + \text{Recall})}$$

1. If we have a problem statement where the **FP** and **FN** are both equally important, we select **Beta = 1**
2. In scenarios where the **FP** is having greater impact than the **FN**, in such case we reduce the **Beta** value between 0 to 1
3. In scenarios where the **FN** is having greater impact than the **FP**, in such case we increase the **Beta** value greater than 1 and might range between 1 to 10

Các thước đo phân loại trong học máy

		Predicted class	
		Positive	Negative
Actual class	Positive	TP	FN
	Negative	FP	TN

$$\text{Precision} = \frac{True\ Positive}{True\ Positive + False\ Positive}$$

$$\text{Recall} = \frac{True\ Positive}{True\ Positive + False\ Negative}$$

$$F_1 = 2 * \frac{precision * recall}{precision + recall}$$

Ma trận nhầm lẫn là một bảng có sự kết hợp của các giá trị dự đoán và thực tế.

Dương tính thực sự (TP). Chúng tôi đã dự đoán dương tính và điều đó là đúng. **Âm tính thật (TN).** Chúng tôi đã dự đoán là âm tính và điều đó đúng. **Dương tính giả: (FP).** Chúng tôi dự đoán là dương tính và dự đoán là sai. **Sai Âm tính: (FN).** Chúng tôi đã dự đoán là âm tính và nó sai.

Độ chính xác giải thích có bao nhiêu trường hợp được dự đoán chính xác thực sự là dương tính.

Nhớ lại giải thích có bao nhiêu trường hợp dương tính thực tế mà chúng tôi có thể dự đoán chính xác bằng mô hình của mình.

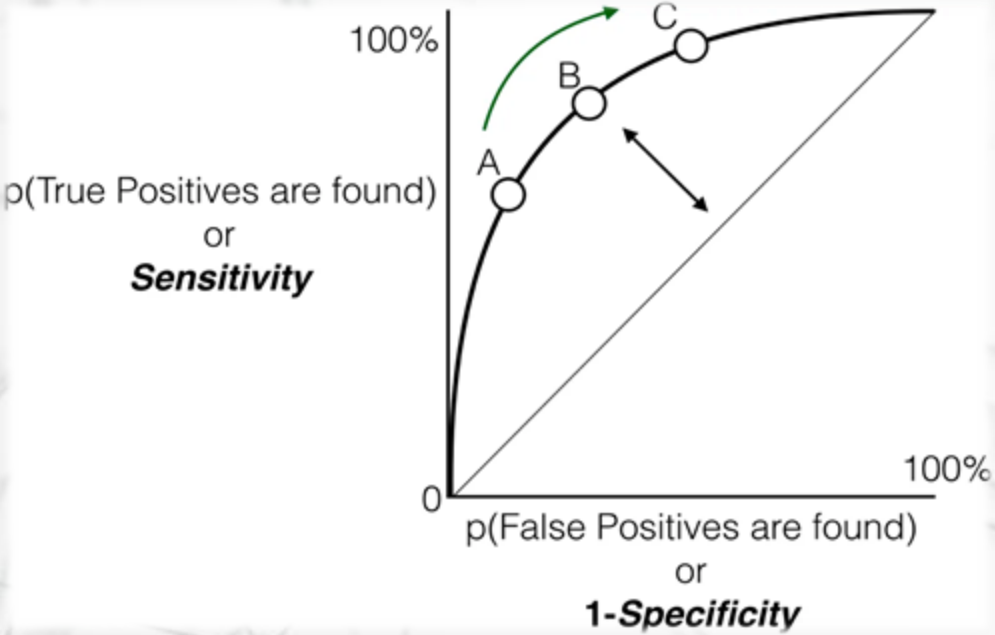
Điểm F1 là giá trị trung bình hài hòa của độ chính xác và khả năng thu hồi. Điểm F1 có thể là thước đo đánh giá hiệu quả trong các trường hợp sau:

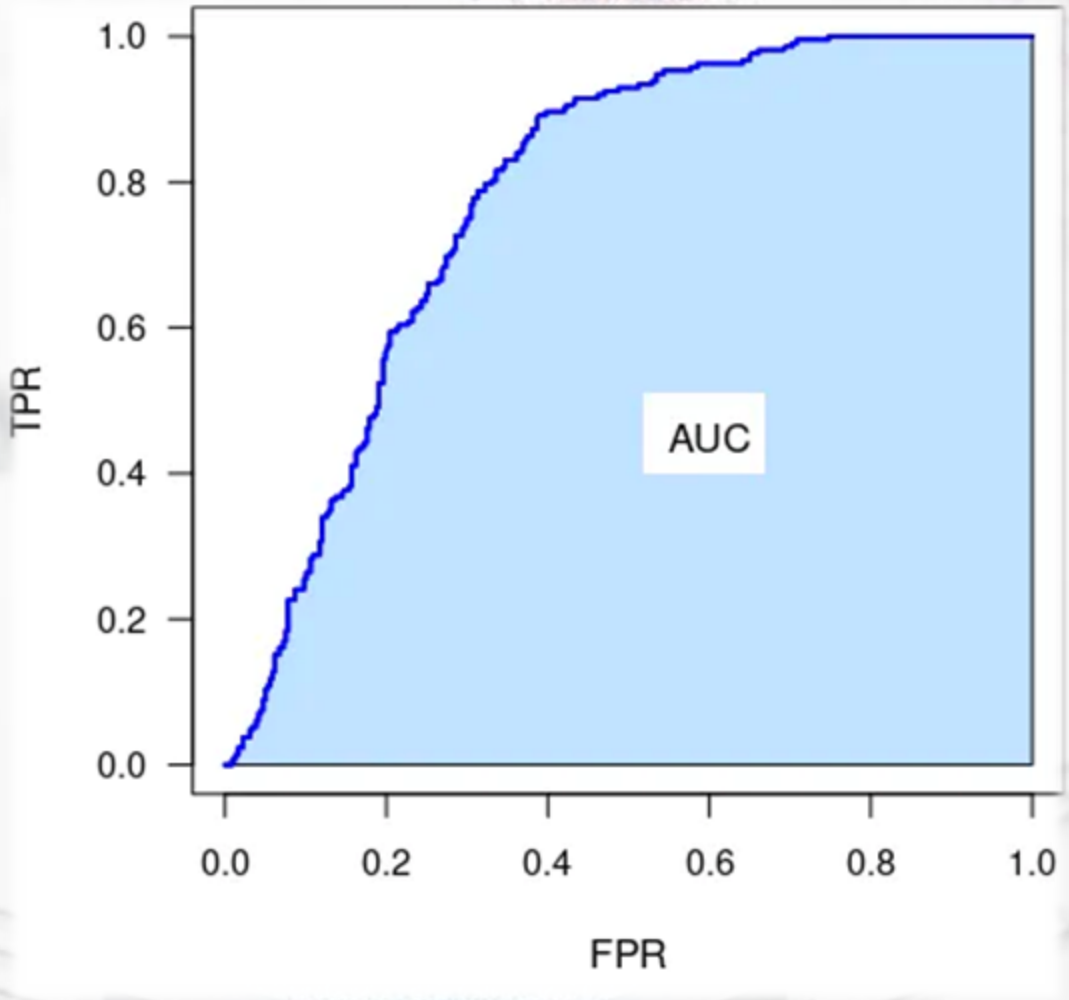
- Khi FP và FN có giá trị như nhau.
- Việc thêm nhiều dữ liệu không làm thay đổi kết quả một cách hiệu quả - TN cao

Classification Metrics in Machine Learning

ROC / AUC. Biểu đồ Đặc tính Toán tử Máy thu (ROC) cung cấp một cách tinh tế để trình bày nhiều ma trận nhầm lẫn được tạo ra ở các ngưỡng khác nhau. ROC biểu thị mối quan hệ giữa tỷ lệ dương tính thực (**TPR**) và tỷ lệ dương tính giả (**FPR**).

TPR* = *Nhớ lại* = *Độ nhạy* = *TP/ (TP+ FN)
***FPR* = 1 – tính đặc hiệu = FP/ (FP+TN)**





Thuật toán phân loại

Các loại	Chung Thuật toán phân loại
Bộ phân loại tuyến tính	• **Hồi quy logistic:** Một mô hình thống kê dự đoán xác suất xảy ra một kết quả phân loại. • **Máy vectơ hỗ trợ (SVM):** Một tập hợp các máy được giám sát phương pháp học tập tạo ra siêu phẳng để tách các điểm dữ liệu thành các lớp khác nhau.
Quyết định Cây và quần thể	• **Cây quyết định:** Một mô hình dạng cây trong đó mỗi nút bên trong biểu thị một phép thử trên một thuộc tính, mỗi nhánh biểu thị kết quả của phép thử và mỗi nút lá biểu thị một nhãn lớp. • **Rừng ngẫu nhiên:** Một tập hợp các cây quyết định, trong đó mỗi cây được xây dựng trên một tập hợp con ngẫu nhiên gồm dữ liệu và đặc điểm. • **Tăng cường độ dốc Máy (GBM):** Một phương pháp tập hợp để xây dựng mô hình tuần tự, mỗi lần sửa các lỗi của mô hình trước đó.
Vịnh ngây thơ	• **Ngây thơ Bayes:** Bộ phân loại xác suất dựa trên định lý Bayes, giả sử sự độc lập giữa các đặc điểm.
K-Hàng xóm gần nhất (KNN)	• **K-Láng giềng gần nhất:** Thuật toán phân loại phi tham số gán một lớp cho một điểm dữ liệu dựa trên trên lớp đa số của k hàng xóm gần nhất của nó.
Mạng thần kinh	• **Mạng thần kinh nhân tạo (ANN):** Một mô hình tính toán lấy cảm hứng từ bộ não con người, bao gồm các nút được kết nối với nhau (tế bào thần kinh) được tổ chức theo lớp. • **Mạng thần kinh chuyển đổi (CNN):** ANN chuyên dụng để xử lý và phân tích dữ liệu hình ảnh. • **Tái phát Mạng thần kinh (RNN):** ANN được thiết kế để xử lý dữ liệu tuần tự, chẳng hạn như văn bản hoặc chuỗi thời gian.

K-Hàng xóm gần nhất

K-Nearest Neighbors (KNN) là một thuật toán học máy được giám sát đơn giản nhưng hiệu quả giúp phân loại hoặc dự đoán các điểm dữ liệu dựa trên mức độ gần nhau của chúng với các ví dụ lân cận trong dữ liệu huấn luyện.

Hoạt động của K-NN có thể được giải thích dựa trên thuật toán dưới đây:

- Bước 1: Chọn số K của hàng xóm
- Bước 2: Tính khoảng cách Euclide của K số láng giềng
- Bước 3: Lấy K hàng xóm gần nhất theo khoảng cách Euclide được tính toán.
- Bước 4: Trong số k hàng xóm này, đếm số lượng điểm dữ liệu trong mỗi danh mục.
- Bước 5: Gán các điểm dữ liệu mới cho danh mục đó mà số lượng điểm lân cận là tối đa.
- Bước 6: Mô hình của chúng tôi đã sẵn sàng.

Thông số chính:

- **k-value:** Số lượng láng giềng được xem xét để phân loại hoặc dự đoán. Giá trị k nhỏ hơn có thể dẫn đến trang bị quá mức, trong khi giá trị k lớn hơn có thể dẫn đến trang bị thiếu.
- **Số liệu khoảng cách:** Phương pháp dùng để tính khoảng cách giữa các điểm dữ liệu. Các số liệu phổ biến bao gồm khoảng cách Euclide, khoảng cách Manhattan và khoảng cách Minkowski.

Công thức đo khoảng cách	
Khoảng cách Euclide (p=2)	$d(x,y) = \sqrt{\sum_{i=1}^{n}(y_i - x_i)^2}$
Khoảng cách Manhattan (p=1)	$\text{Manhattan Distance} = d(x,y) = \left(\sum_{i=1}^{m}\lvert x_i - y_i\rvert\right)$
Khoảng cách Minkowski	$\text{Minkowski Distance} = \left(\sum_{i=1}^{n}\lvert x_i - y_i\rvert\right)^{1/p}$
Khoảng cách Hamming	$\text{Hamming Distance} = D_H = \left(\sum_{i=1}^{k}\lvert x_i - y_i\rvert\right)$ $x=y \quad D=0$ $x\neq y \quad D\neq 1$

K-Hàng xóm gần nhất

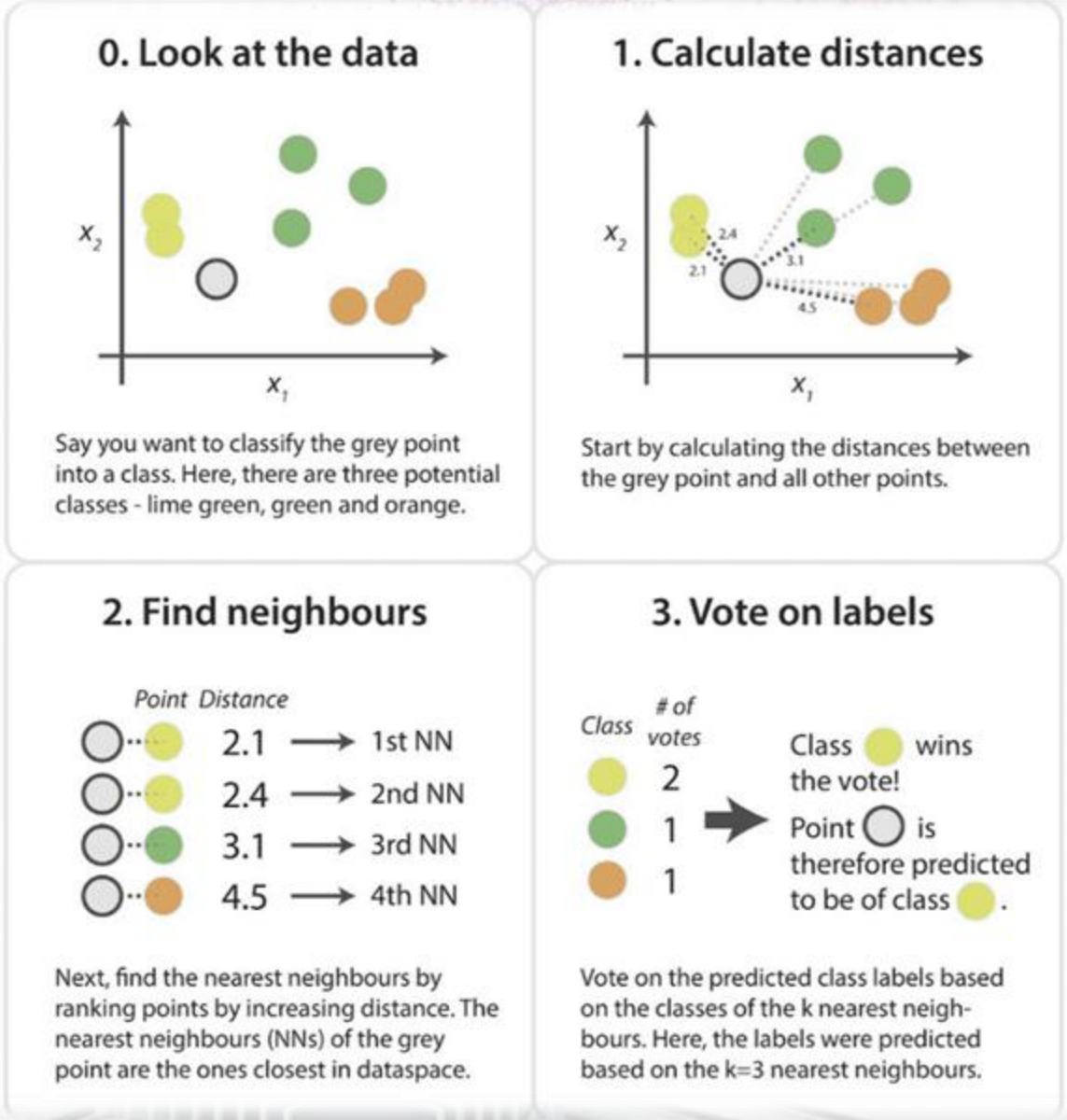
K-Nearest Neighbors (KNN) là một thuật toán học máy được giám sát đơn giản nhưng hiệu quả giúp phân loại hoặc dự đoán các điểm dữ liệu dựa trên mức độ gần nhau của chúng với các ví dụ lân cận trong dữ liệu huấn luyện.

Hoạt động của K-NN có thể được giải thích dựa trên thuật toán dưới đây:

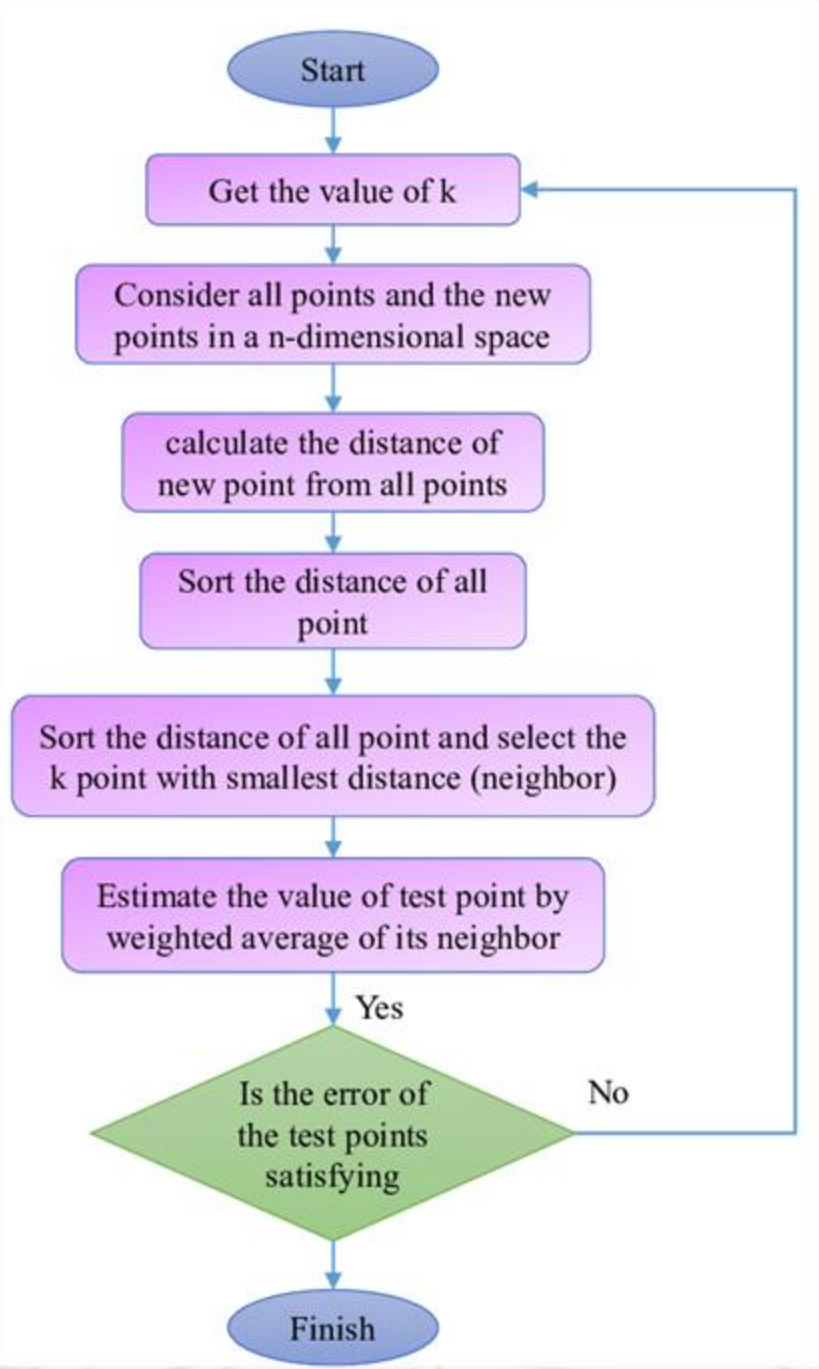
- Bước 1: Chọn số K của hàng xóm
- Bước 2: Tính khoảng cách Euclide của K số láng giềng
- Bước 3: Lấy K hàng xóm gần nhất theo khoảng cách Euclide được tính toán.
- Bước 4: Trong số k hàng xóm này, đếm số lượng điểm dữ liệu trong mỗi danh mục.
- Bước 5: Gán các điểm dữ liệu mới cho danh mục đó mà số lượng điểm lân cận là tối đa.
- Bước 6: Mô hình của chúng tôi đã sẵn sàng.

Thông số chính:

- **k-value:** Số lượng láng giềng được xem xét để phân loại hoặc dự đoán. Giá trị k nhỏ hơn có thể dẫn đến trang bị quá mức, trong khi giá trị k lớn hơn có thể dẫn đến trang bị thiếu.
- **Số liệu khoảng cách:** Phương pháp dùng để tính khoảng cách giữa các điểm dữ liệu. Các số liệu phổ biến bao gồm khoảng cách Euclide, khoảng cách Manhattan và khoảng cách Minkowski.



K-Hàng xóm gần nhất




```python
# --- 1. Define Euclidean Distance ---
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2)**2))

# --- 2. Implement KNN Classifier from Scratch ---
class MyKNNClassifier:
    def __init__(self, n_neighbors=5):
        self.n_neighbors = n_neighbors

    def fit(self, X, y):
        # KNN is a lazy learner, so 'fitting' just means storing the data
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        predictions = [self._predict_single_sample(x) for x in X]
        return np.array(predictions)

    def _predict_single_sample(self, x):
        # Calculate distances from x to all training samples
        distances = [euclidean_distance(x, train_point) for train_point in self.X_train]

        # Get the indices of the k-nearest neighbors
        k_indices = np.argsort(distances)[:self.n_neighbors]

        # Get the labels of the k-nearest neighbors
        k_nearest_labels = [self.y_train[i] for i in k_indices]

        # Return the most common label among the k neighbors (majority vote)
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]

```

K-Hàng xóm gần nhất

Ưu điểm của KNN:

- **Đơn giản:** Dễ hiểu và dễ thực hiện.
- **Phi tham số:** Không đưa ra giả định về phân bổ dữ liệu.
- **Tính linh hoạt:** Có thể được sử dụng cho cả phân loại và hồi quy.
- **Độ chính xác:** Có thể đạt được độ chính xác cao đối với dữ liệu có cấu trúc tốt.

Nhược điểm của KNN:

- **Chi phí tính toán:** Có thể chậm đối với các tập dữ liệu lớn.
- **Độ nhạy với giá trị k:** Việc chọn đúng giá trị k là rất quan trọng.
- **Lời nguyền của chiều:** Có thể kém hiệu quả hơn trong không gian nhiều chiều.
- **Nhạy cảm với tiếng ồn:** Dữ liệu nhiễu có thể ảnh hưởng đến kết quả.

Bộ dữ liệu mống mắt

→https://scikit-learn.org/stable/auto_examples/datasets/plot_iris_dataset.html#sphx-glr-auto-examples-datasets-plot-iris-dataset-py

Ứng dụng của KNN:

- **Nhận dạng hình ảnh:** Phân loại hình ảnh dựa trên đặc điểm tương tự.
- **Hệ thống đề xuất:** Đề xuất các mục dựa trên sở thích của người dùng.
- **Phân khúc khách hàng:** Phân nhóm khách hàng dựa trên hành vi của họ.
- **Chẩn đoán y khoa:** Dự đoán bệnh dựa trên dữ liệu bệnh nhân.

Trong scikit-learn,

1- sklearn.neighbors

→https://scikit-learn.org/stable/modules/neighbors.html#

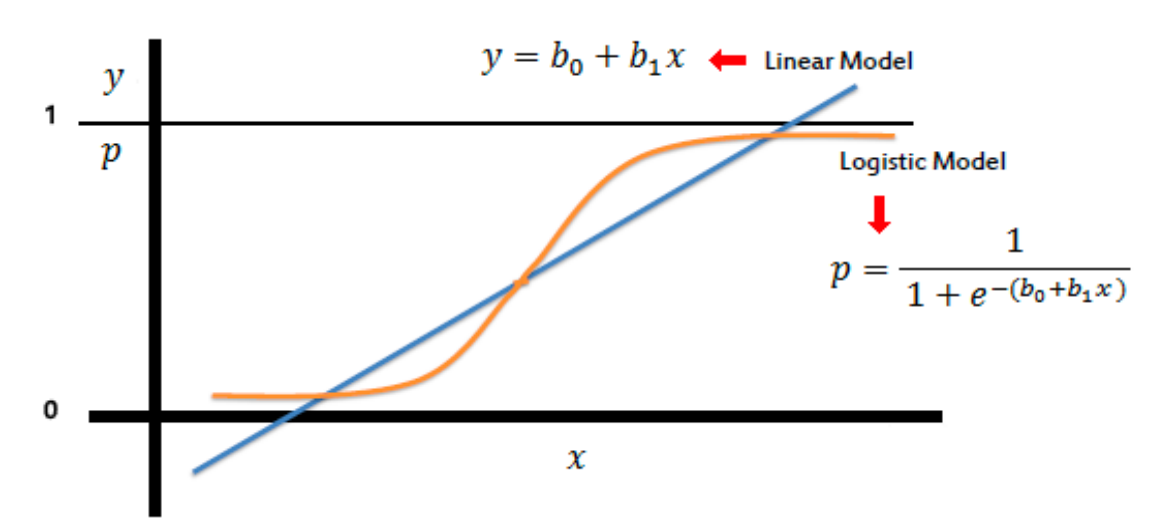
→https://scikit-learn.org/stable/auto_examples/neighbors/plot_nearest_centroid.html

2- Ví dụ: cách sử dụng KNeighborsClassifier.

→https://scikit-learn.org/stable/auto_examples/neighbors/plot_classification.html#sphx-glr-auto-examples-hàng xóm-plot-classification- py

Hồi quy logistic

Hồi quy logistic là mô hình thống kê được sử dụng để dự đoán xác suất xảy ra kết quả nhị phân (ví dụ: có/không, đúng/sai, 1/0). Đó là một kỹ thuật được sử dụng rộng rãi trong nhiều lĩnh vực khác nhau, bao gồm học máy, thống kê và khoa học dữ liệu.



Trong scikit-learn,

- lớp ***sklearn.Tuyến_model.LogisticRegression***

→https://scikit-learn.org/stable/modules/generated/sklearn.Tuyến_model.LogisticRegression.html#sklearn.Tuyến_model.LogisticRegression

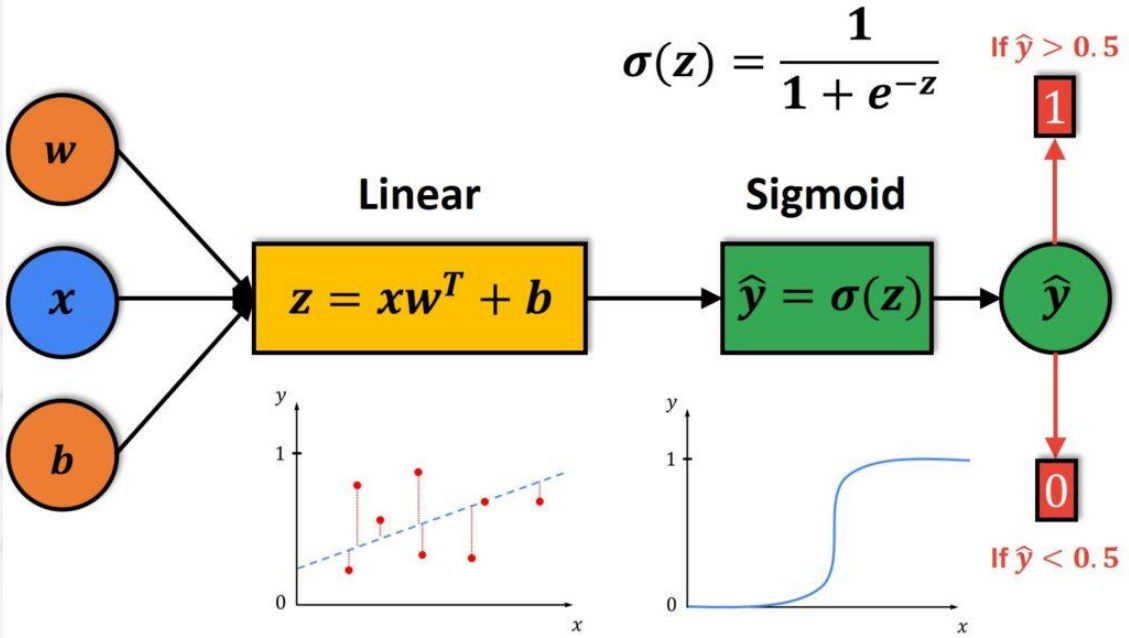
- **Ví dụ:** Bộ phân loại 3 lớp hồi quy logistic

→https://scikit-learn.org/stable/auto_examples/Tuyến_model/plot_iris_logic.html#sphx-glr-auto-examples-tuyến tính-model-plot-iris-logic-py

Ứng dụng hồi quy logistic:

- Dự đoán tỷ lệ rời bỏ khách hàng: Xác định liệu khách hàng có khả năng ngừng sử dụng sản phẩm, dịch vụ hay không.
- Chấm điểm tín dụng: Đánh giá rủi ro vỡ nợ cho vay.
- Chẩn đoán y khoa: Dự đoán sự hiện diện hay vắng mặt của bệnh. - Lọc thư rác qua email: Nhận dạng các email spam dựa trên nội dung của chúng.
- Phân tích tình cảm: Xác định tình cảm (tích cực, tiêu cực hoặc trung tính) của một văn bản.

Hồi quy logistic




```
Input: Training data (X, y), learning rate α, number of iterations

1. Initialize weights w and bias b
2. For each iteration:
   a. Compute z = X.w + b
   b. Compute prediction y_hat = sigmoid(z)
   c. Compute loss = cross-entropy(y, y_hat)
   d. Compute gradients dw, db
   e. Update parameters: w := w − α * dw; b := b − α * db
3. Return trained weights w, b
```

Hồi quy logistic hoạt động như thế nào:

- **Đầu vào:** Mô hình lấy một tập hợp các biến dự đoán (đặc điểm) làm đầu vào.
- **Kết hợp tuyến tính:** Mô hình tính toán sự kết hợp tuyến tính của các đặc điểm đầu vào, được tính trọng số bởi các hệ số.
- **Hàm sigmoid:** Kết quả của tổ hợp tuyến tính được truyền qua hàm sigmoid (còn được gọi là hàm logistic). Hàm này ánh xạ bất kỳ giá trị thực nào tới giá trị từ 0 đến 1.
- **Ước tính xác suất:** Đầu ra của hàm sigmoid được hiểu là xác suất của kết quả dương tính.

Hàm sigmoid:

$$sigmoid(x) = 1 / (1 + e^{(-x)})$$

trong đó: x là tổ hợp tuyến tính của các đặc tính đầu vào. e là số Euler (xấp xỉ 2,71828).

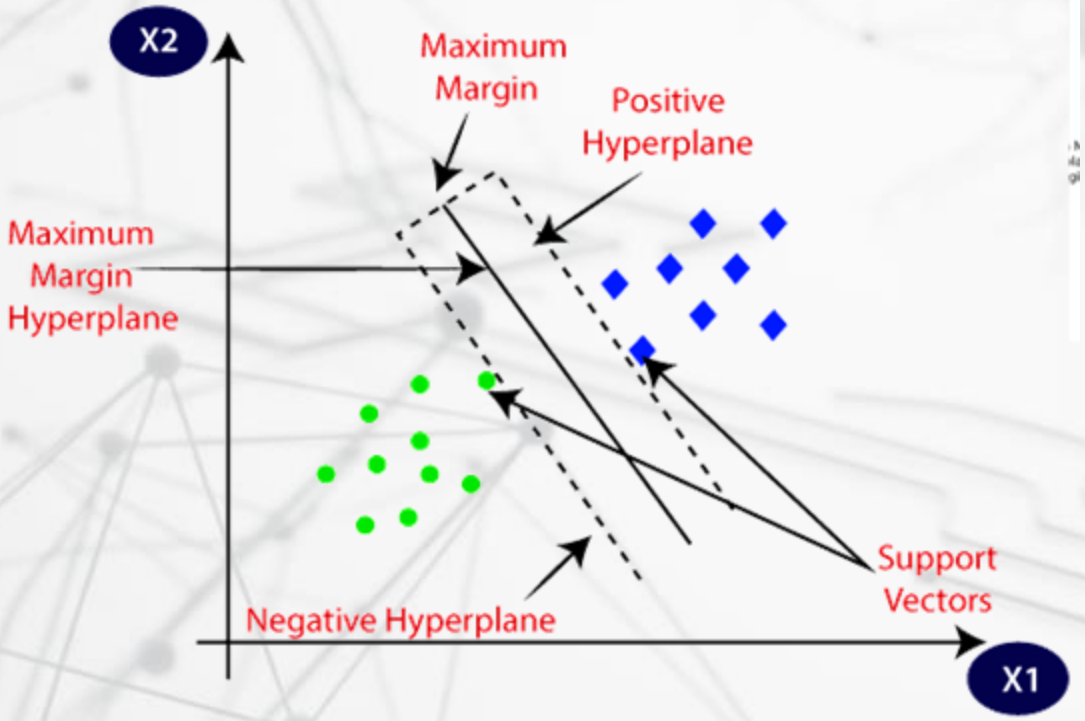
Hồi quy logistic

Ưu điểm	Nhược điểm
Đơn giản và có thể giải thích được	Giả sử tuyến tính giữa dự báo và tỷ lệ cược log
Tính toán hiệu quả	Giới hạn cho các vấn đề phức tạp, phi tuyến tính
Cung cấp ước tính xác suất	Nhạy cảm với các ngoại lệ
Xử lý các loại dữ liệu khác nhau	Yêu cầu mẫu lớn kích thước
Ít bị trang bị quá mức (với tính năng chính quy hóa)	Vấn đề đa cộng tuyến
Tốt như một cơ sở người mẫu	Chỉ dành cho những kết quả rời rạc
Được sử dụng và hiểu rộng rãi	Có thể không phù hợp với bộ dữ liệu chiều cao

Máy vectơ hỗ trợ (SVM)

Máy vectơ hỗ trợ (SVM) là một thuật toán học máy mạnh mẽ được sử dụng cho các nhiệm vụ phân loại và hồi quy. Chúng đặc biệt hiệu quả trong không gian nhiều chiều và được biết đến với khả năng xử lý các ranh giới quyết định phức tạp.

Thuật toán SVM có thể được sử dụng để nhận diện khuôn mặt, phân loại hình ảnh, phân loại văn bản, v.v.



Siêu phẳng: Trong SVM, siêu phẳng là ranh giới quyết định tối ưu giúp phân tách các lớp một cách tốt nhất. Hình dạng của nó phụ thuộc vào số lượng đặc điểm: với 2 đặc điểm, nó là một đường thẳng, với 3 đặc điểm, nó là một mặt phẳng và ở các chiều cao hơn, nó trở thành bề mặt n chiều.

Vectơ hỗ trợ: Các điểm dữ liệu hoặc vectơ gần nhất với siêu phẳng và ảnh hưởng đến vị trí của siêu phẳng được gọi là Vector hỗ trợ. Vì các vectơ này hỗ trợ siêu phẳng nên được gọi là vectơ hỗ trợ.

Ý tưởng cốt lõi: SVM tìm thấy một **siêu phẳng tối đa hóa lề** giữa hai lớp. Mục tiêu tối ưu hóa là:

$$\min_{w,b} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(w^T x_i + b))$$

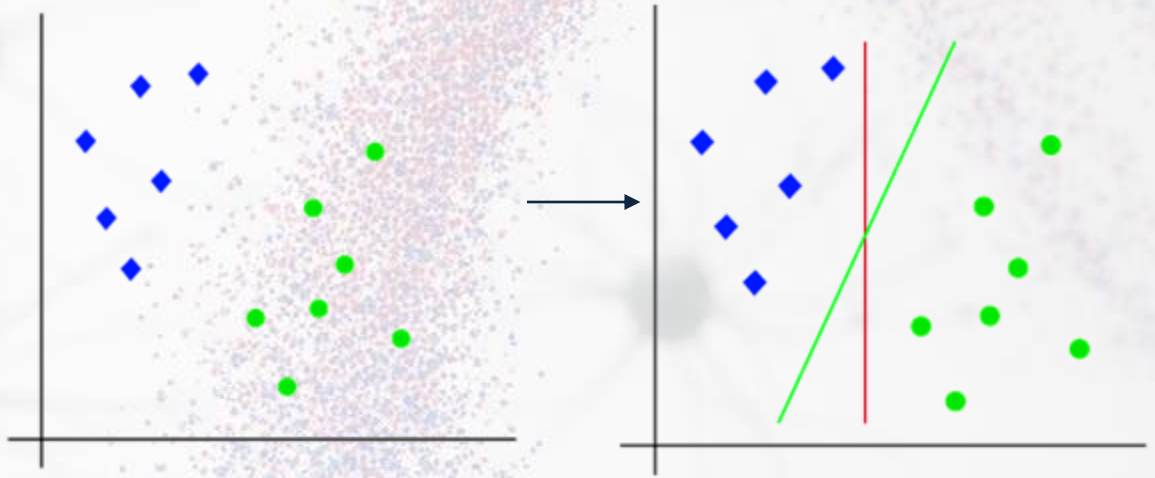
Where:

- w: weight vector
- b: bias
- C: regularization parameter (often written as $\frac{1}{\lambda}$)
- $y_i \in \{-1, +1\}$: binary class labels

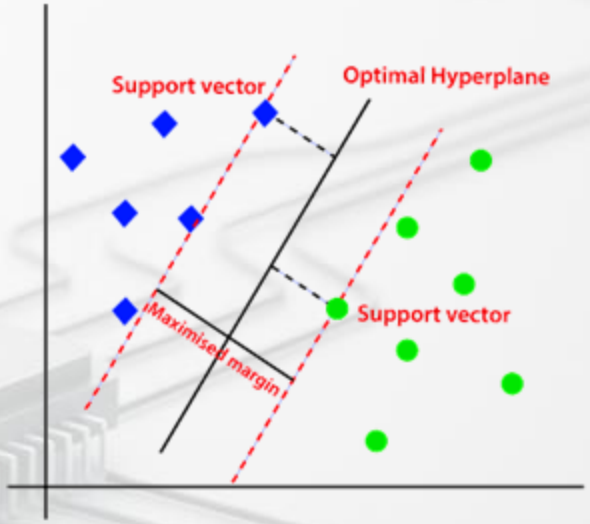
Máy vectơ hỗ trợ (SVM)

SVM hoạt động như thế nào:

1. **Ánh xạ tính năng:** SVM ánh xạ dữ liệu đầu vào vào không gian tính năng có chiều cao hơn. Ánh xạ này có thể là tuyến tính hoặc phi tuyến tính, tùy thuộc vào hàm kernel được sử dụng.
2. **Tách siêu phẳng:** SVM tìm siêu phẳng tối ưu (ranh giới quyết định) phân tách các điểm dữ liệu thành các lớp khác nhau với lề tối đa. Lề này là khoảng cách giữa siêu phẳng và các điểm dữ liệu gần nhất, được gọi là vectơ hỗ trợ.



3. **Phân loại:** Các điểm dữ liệu mới được phân loại dựa trên phía nào của siêu phẳng mà chúng rơi vào.



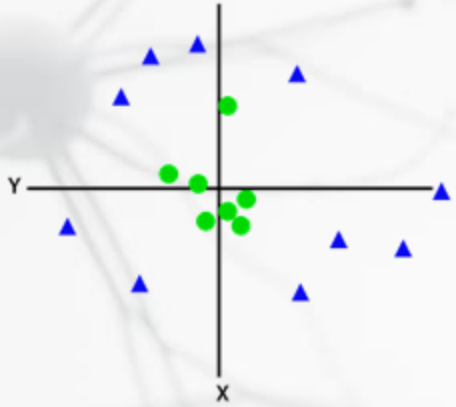
Các hàm hạt nhân:

Việc lựa chọn hàm kernel xác định kiểu ánh xạ vào không gian đặc trưng có chiều cao hơn. Các hàm kernel phổ biến bao gồm:

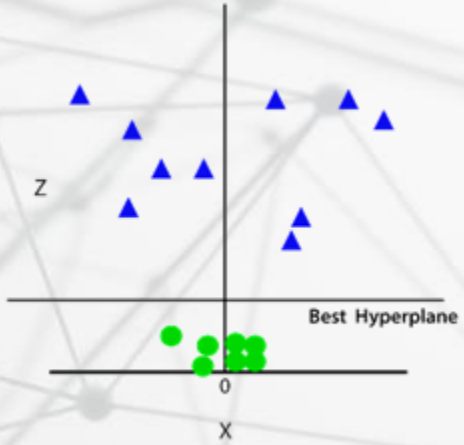
- **Hạt nhân tuyến tính:** Một hạt nhân đơn giản ánh xạ dữ liệu một cách tuyến tính.
- **Hạt nhân đa thức:** Hạt nhân đưa các thuật ngữ đa thức vào không gian đặc trưng.
- **Hạt nhân hàm cơ sở xuyên tâm (RBF):** Hạt nhân phi tuyến tính ánh xạ dữ liệu vào không gian đặc trưng vô hạn chiều.

SVM phi tuyến tính

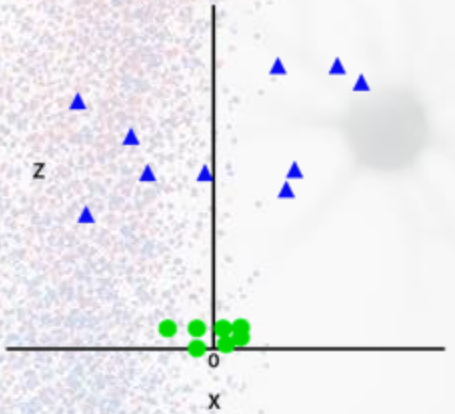
Nếu dữ liệu được sắp xếp tuyến tính thì chúng ta có thể phân tách nó bằng cách sử dụng một đường thẳng, nhưng đối với dữ liệu phi tuyến tính, chúng ta không thể vẽ một đường thẳng duy nhất.



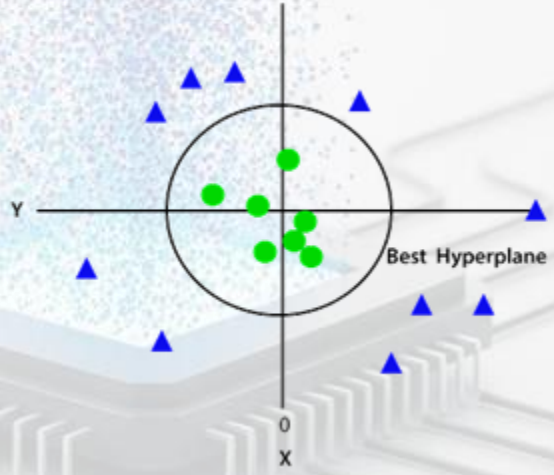
Vì vậy, bây giờ SVM sẽ chia tập dữ liệu thành các lớp theo cách sau



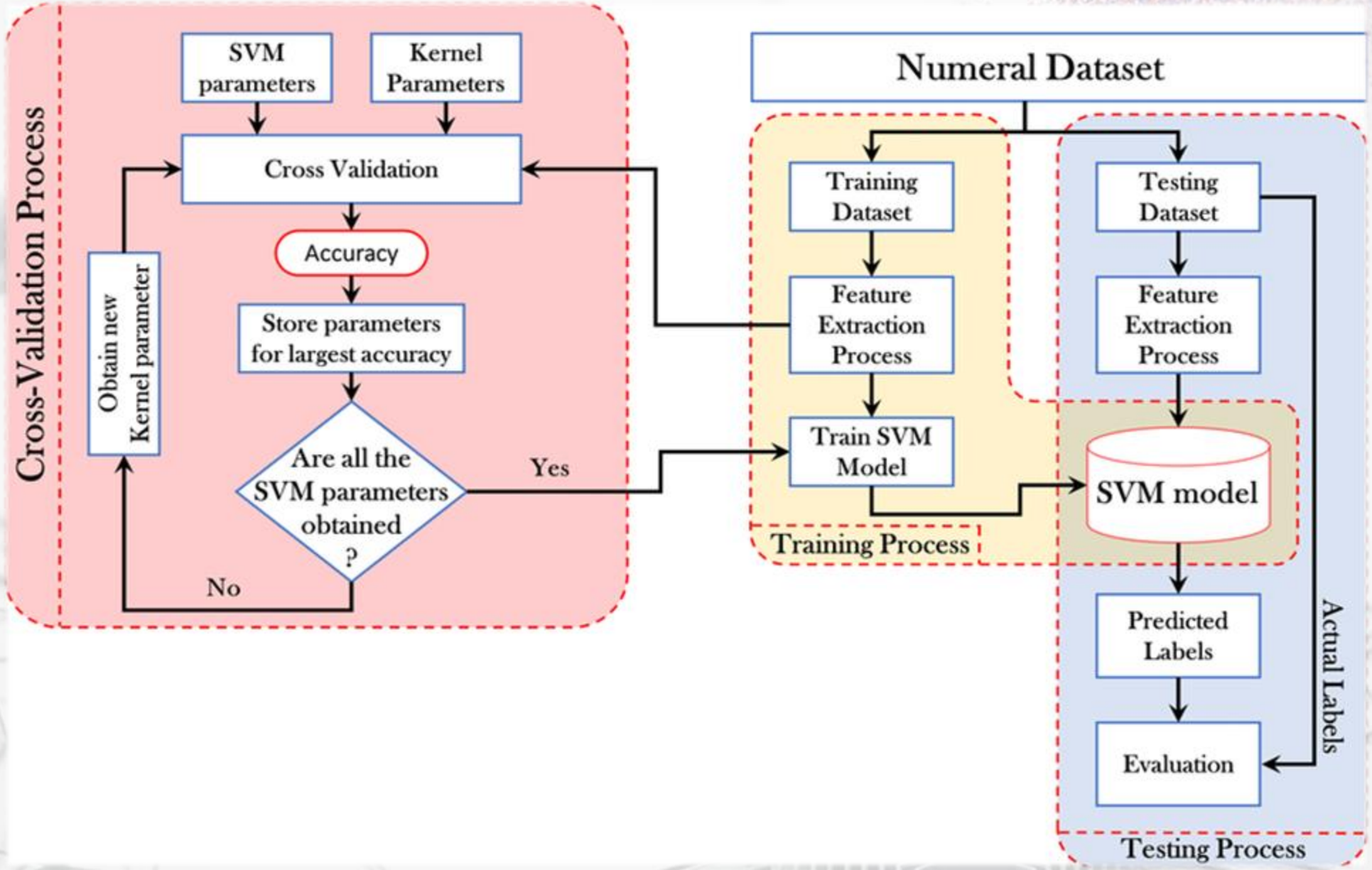
Vì vậy, để tách các điểm dữ liệu này, chúng ta cần thêm một chiều nữa. Đối với dữ liệu tuyến tính, chúng tôi đã sử dụng hai chiều x và y, vì vậy đối với dữ liệu phi tuyến tính, chúng tôi sẽ thêm chiều thứ ba z. Nó có thể được tính như sau: *z=x2 +y2*



Vì chúng ta ở trong Không gian 3 chiều nên nó trông giống như một mặt phẳng song song với trục x. Nếu chúng ta chuyển đổi nó trong không gian 2d với z=1 thì nó sẽ trở thành



Schematic diagram for SVM



Xây dựng và huấn luyện SVM tuyến tính

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

# Step 1: Generate simple binary data
X, y = make_blobs(n_samples=200, centers=2, random_state=42)
y = np.where(y == 0, -1, 1)  # convert to -1, +1 for SVM

# Step 2: Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 3: Define SVM class
class SVM:
    def __init__(self, lr=0.001, lambda_param=0.01, n_iters=1000):
        self.lr = lr
        self.lambda_param = lambda_param
        self.n_iters = n_iters
        self.w = None
        self.b = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0

        for _ in range(self.n_iters):
            for idx, x_i in enumerate(X):
                if y[idx] * (np.dot(x_i, self.w) + self.b) >= 1:
                    self.w -= self.lr * (2 * self.lambda_param * self.w)
                else:
                    self.w -= self.lr * (2 * self.lambda_param * self.w - y[idx] * x_i)
                    self.b -= self.lr * y[idx]

    def predict(self, X):
        return np.sign(np.dot(X, self.w) + self.b)

# Step 4: Train and evaluate
model = SVM()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
accuracy = np.mean(predictions == y_test)

print(f"Test Accuracy: {accuracy:.2%}")
```

Trong Scikit-learn, SVM để phân loại - **SVC**, **NuSVC** và **LinearSVC**

→https://scikit-learn.org/stable/modules/svm.html#

Ví dụ:

1. Vẽ các trình phân loại SVM khác nhau trong tập dữ liệu iris →https://scikit-learn.org/stable/auto_examples/svm/plot_iris_svc.html#sphx-glr-auto-examples-svm-plot-iris-svc-py

2. SVM với kernel tùy chỉnh

→https://scikit-learn.org/stable/auto_examples/svm/plot_custom_kernel.html#sphx-glr-auto-examples-svm-plot-custom-kernel-py

3. Thông số RBF SVM

→https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

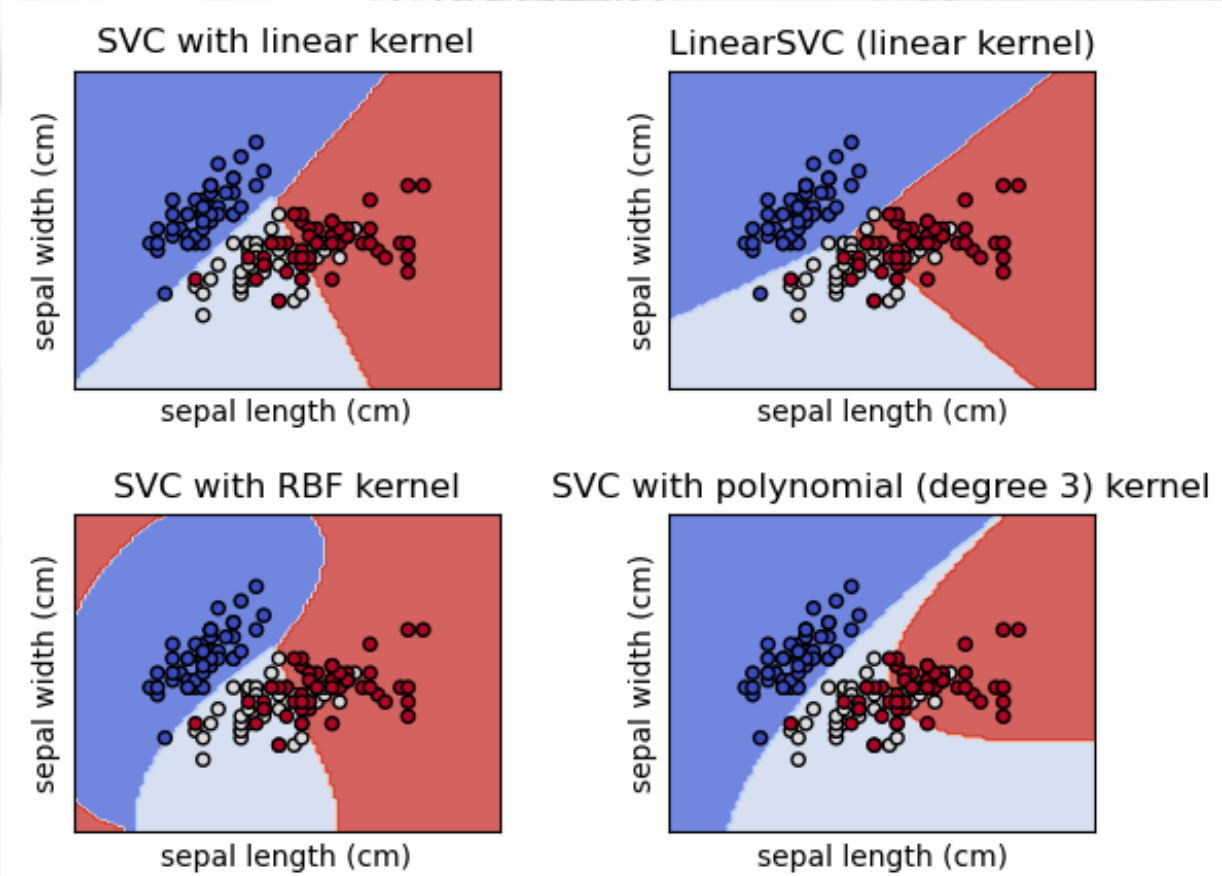
Máy vectơ hỗ trợ (SVM)

Ưu điểm của SVM:

- **Hiệu quả trong không gian nhiều chiều:** SVM có thể xử lý các ranh giới quyết định phức tạp trong dữ liệu nhiều chiều.
- **Mạnh mẽ với các ngoại lệ:** SVM ít nhạy cảm hơn với các ngoại lệ do chúng tập trung vào các vectơ hỗ trợ.
- **Đa năng:** SVM có thể được sử dụng cho cả nhiệm vụ phân loại và hồi quy.
- **Hiệu quả:** SVM có thể hiệu quả đối với các tập dữ liệu lớn, đặc biệt khi sử dụng các thủ thuật kernel.

Nhược điểm của SVM:

- **Độ phức tạp tính toán:** Việc đào tạo SVM có thể tốn kém về mặt tính toán đối với các tập dữ liệu lớn, đặc biệt là với các hạt nhân phi tuyến tính.
- **Lựa chọn hạt nhân:** Việc chọn chức năng hạt nhân thích hợp có thể là một thách thức.
- **Độ nhạy cảm với siêu tham số:** SVM có siêu tham số (như hàm kernel và tham số chính quy) cần được điều chỉnh để có hiệu suất tối ưu.



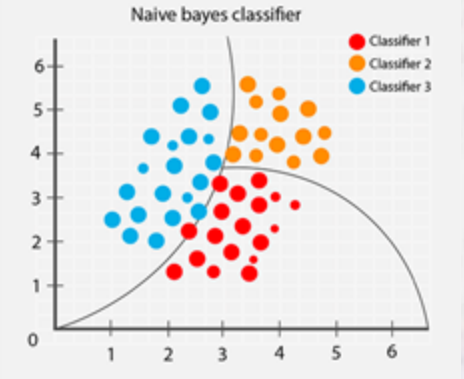
Phân loại Naive Bayes

Naive Bayes là một thuật toán phân loại xác suất dựa trên định lý Bayes, giả định rằng các đặc điểm là độc lập với lớp. Mặc dù giả định về tính độc lập này thường bị vi phạm trong dữ liệu thực tế nhưng Naive Bayes vẫn có thể hoạt động tốt một cách đáng ngạc nhiên trong nhiều trường hợp.

$$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$$

using Bayesian probability terminology, the above equation can be written as

$$\text{Posterior} = \frac{\text{prior x likelihood}}{\text{evidence}}$$



Naive Bayes hoạt động như thế nào

1. Tính xác suất:

- **Xác suất trước:** Xác suất của mỗi lớp xảy ra độc lập với các đặc điểm.
- **Xác suất có điều kiện:** Xác suất xảy ra một tính năng cho một lớp cụ thể.

1. Áp dụng định lý Bayes:

- Sử dụng định lý Bayes, tính xác suất hậu nghiệm của mỗi lớp dựa trên các đặc điểm được quan sát.
- Lớp có xác suất hậu nghiệm cao nhất được dự đoán là lớp có khả năng xảy ra cao nhất.

Whether	Play
Sunny	No
Sunny	No
Overcast	Yes
Rainy	Yes
Rainy	Yes
Rainy	No
Overcast	Yes
Sunny	No
Sunny	Yes
Rainy	Yes
Sunny	Yes
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table

Whether	No	Yes
Overcast		4
Sunny	2	3
Rainy	3	2
Total	5	9

Likelihood Table 1

Whether	No	Yes		
Overcast		4	=4/14	0.29
Sunny	2	3	=5/14	0.36
Rainy	3	2	=5/14	0.36
Total	5	9		
	=5/14	=9/14		
	0.36	0.64		

Likelihood Table 2

Whether	No	Yes	Posterior Probability for No	Posterior Probability for Yes
Overcast		4	0/5=0	4/9=0.44
Sunny	2	3	2/5=0.4	3/9=0.33
Rainy	3	2	3/5=0.6	2/9=0.22
Total	5	9		

Các loại phân loại chính của Naive Bayes

01	Gaussian Naive Bayes	• Assumes features are normally distributed. • Suitable for continuous numerical data. • Calculates conditional probabilities based on the mean and standard deviation of each feature within each class.
02	Multinomial Naive Bayes	• Designed for count data (e.g., word frequencies in text documents). • Assumes features follow a multinomial distribution. • Calculates conditional probabilities based on the frequency of each feature within each class.
03	Bernoulli Naive Bayes	• Suitable for binary features (e.g., presence or absence of a word in a document). • Assumes features follow a Bernoulli distribution. • Calculates conditional probabilities based on the probability of a feature being present or absent within each class.

Phân loại Naive Bayes

Ưu điểm của Naive Bayes

- **Đơn giản:** Dễ thực hiện và dễ hiểu.
- **Hiệu quả:** Có thể xử lý tập dữ liệu lớn một cách hiệu quả.
- **Mạnh mẽ:** Có thể hoạt động tốt ngay cả với dữ liệu bị nhiễu hoặc bị thiếu.

Nhược điểm của Naive Bayes

- **Giả định độc lập:** Giả định về tính độc lập của đối tượng có thể bị vi phạm trong nhiều tình huống thực tế.

- **Độ nhạy với số lượng 0:** Nếu kết hợp lớp đối tượng không xuất hiện trong dữ liệu huấn luyện thì xác suất có điều kiện sẽ trở thành 0, dẫn đến dự đoán không chính xác.

Ứng dụng của Naive Bayes

- **Phân loại văn bản:** Lọc thư rác, phân tích cảm tính, lập mô hình chủ đề
- **Hệ thống đề xuất:** Đề xuất các mục dựa trên sở thích của người dùng
- **Chẩn đoán y khoa:** Dự đoán bệnh dựa trên triệu chứng
- **Dự báo thời tiết:** Dự báo điều kiện thời tiết

Trong scikit-learn,

→https://scikit-learn.org/1.5/modules/naive_bayes.html

→ví dụ: https://scikit-learn.org/1.5/auto_examples/classification/plot_classifier_comparison.html

Phân loại cây quyết định

Cây quyết định là một thuật toán học máy phổ biến thường được sử dụng cho cả nhiệm vụ phân loại và hồi quy.

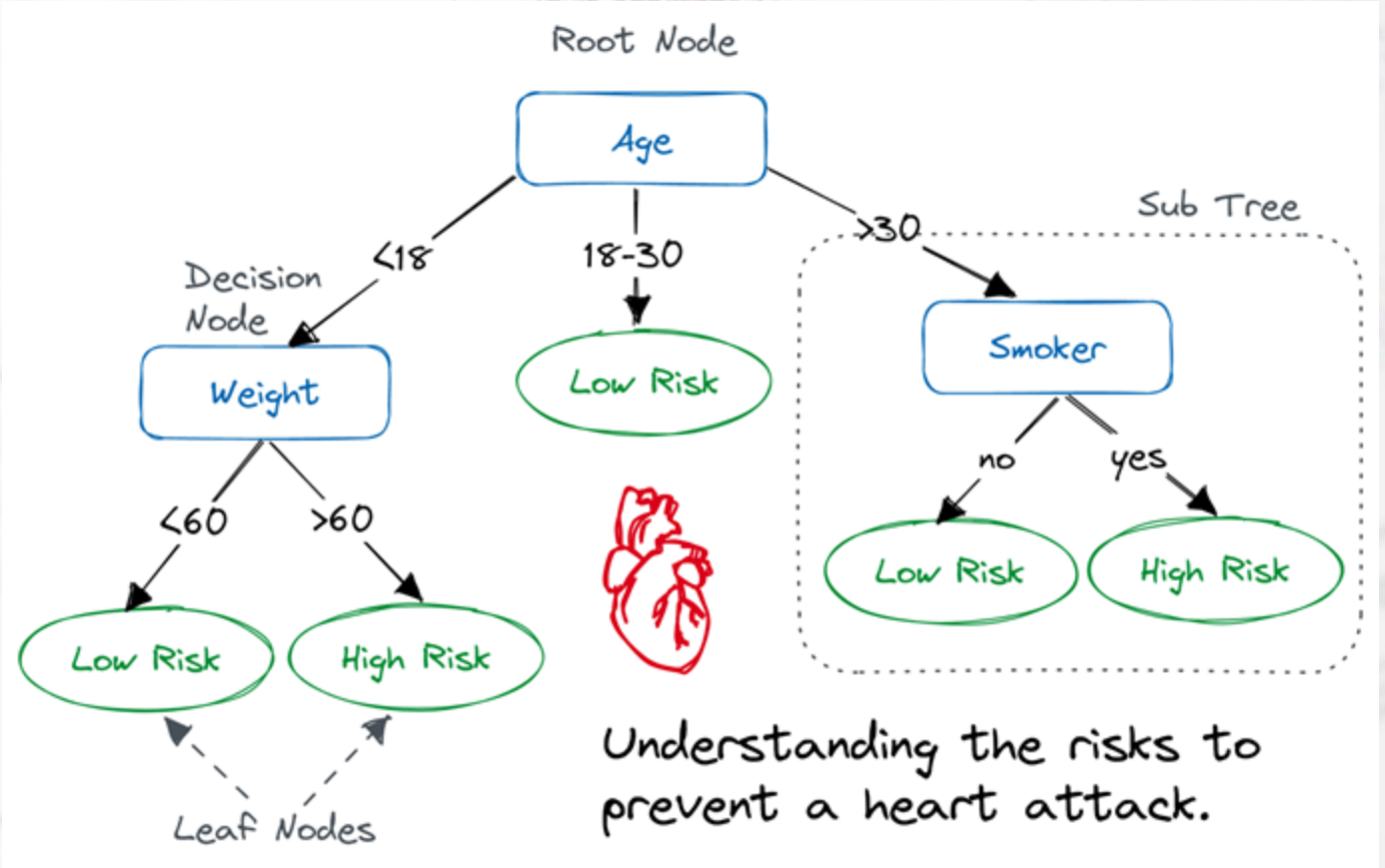
Trong bối cảnh phân loại, họ tạo ra một mô hình dạng cây trong đó mỗi nút bên trong đại diện cho một bài kiểm tra trên một thuộc tính (ví dụ: "Tuổi có lớn hơn 30 không?"), mỗi nhánh đại diện cho các kết quả có thể có của bài kiểm tra và mỗi nút lá đại diện cho một nhãn lớp.

Cây quyết định là một phương pháp không phân phối hoặc không tham số, không phụ thuộc vào các giả định phân phối xác suất.

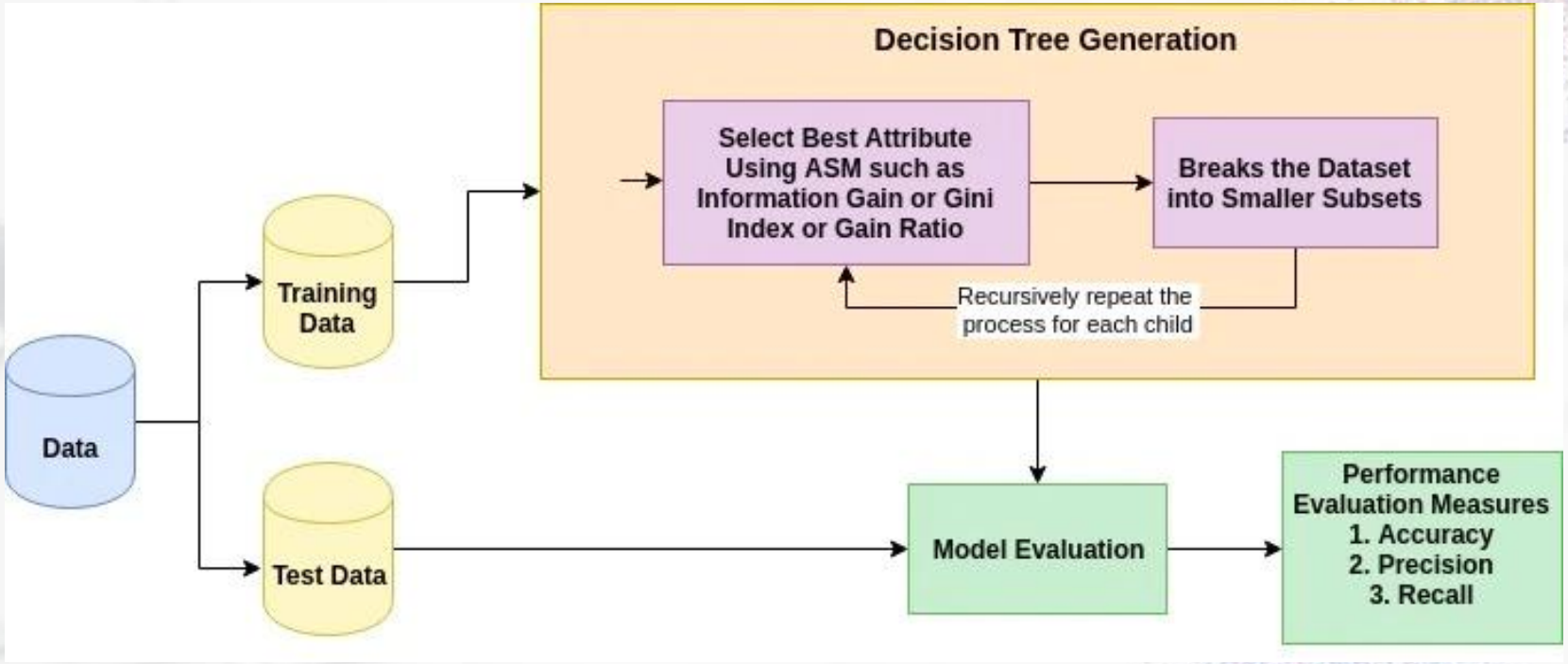
Cây quyết định có thể xử lý dữ liệu nhiều chiều với độ chính xác tốt.

Thực hành:

- →https://www.datacamp.com/tutorial/decision-tree-classclass-python



Thuật toán cây quyết định



Ý tưởng cơ bản đằng sau bất kỳ thuật toán cây quyết định nào như sau:

1. Chọn thuộc tính tốt nhất bằng cách sử dụng Biện pháp lựa chọn thuộc tính (ASM) để phân chia các bản ghi. 2. Biến thuộc tính đó thành nút quyết định và chia tập dữ liệu thành các tập con nhỏ hơn.

3. Bắt đầu xây dựng cây bằng cách lặp lại quy trình này một cách đệ quy cho từng phần tử con cho đến khi một trong các điều kiện phù hợp:

- Tất cả các bộ thuộc về cùng một giá trị thuộc tính.
- Không còn thuộc tính nào còn lại.
- Không còn trường hợp nào nữa.

Phân loại rừng ngẫu nhiên

Random Forest là một phương pháp học tập tổng hợp mạnh mẽ, kết hợp nhiều cây quyết định để đưa ra dự đoán. Nó đặc biệt hiệu quả đối với các nhiệm vụ phân loại do khả năng xử lý các tập dữ liệu lớn, giảm việc trang bị quá mức và cung cấp tầm quan trọng của tính năng.

Rừng ngẫu nhiên hoạt động như thế nào?

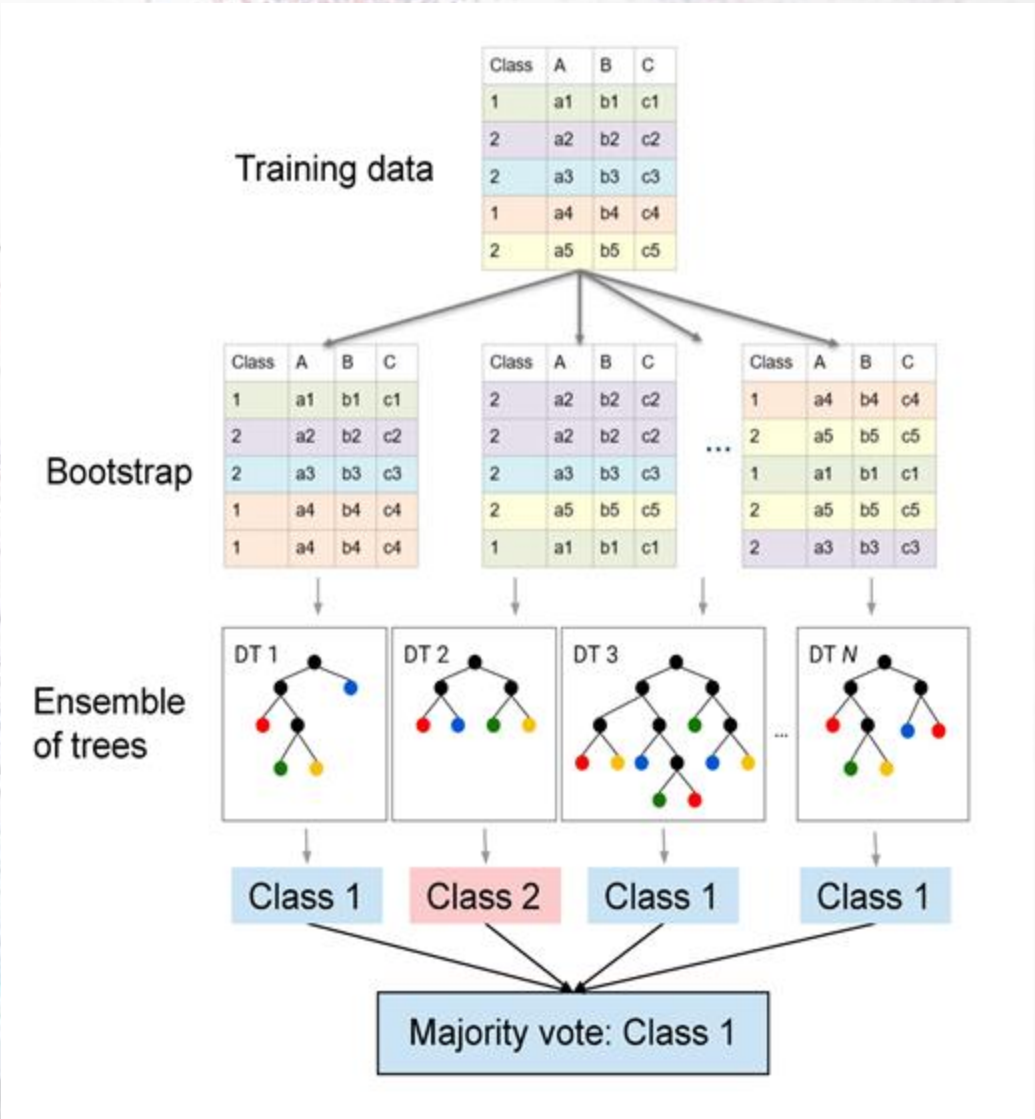
- **Lấy mẫu Bootstrap:** Thuật toán chọn ngẫu nhiên các tập hợp con dữ liệu từ tập dữ liệu gốc để thay thế. Điều này tạo ra nhiều "mẫu bootstrap."
- **Tăng trưởng cây quyết định:** Mỗi mẫu bootstrap được sử dụng để phát triển cây quyết định. Cây được phát triển đến kích thước tối đa mà không cần cắt tỉa.
- **Dự đoán:** Để đưa ra dự đoán cho một điểm dữ liệu mới, thuật toán sẽ chuyển nó qua từng cây quyết định và thu thập lớp dự đoán từ mỗi cây quyết định.
- **Biểu quyết:** Lớp thường xuyên nhất trong số tất cả các dự đoán từ các cây riêng lẻ sẽ trở thành dự đoán cuối cùng.

Các thông số chính trong Rừng ngẫu nhiên

- **Số lượng cây:** Số lượng cây quyết định trong rừng.
- **Độ sâu tối đa:** Độ sâu tối đa của từng cây quyết định.
- **Phân chia mẫu tối thiểu:** Số lượng mẫu tối thiểu cần thiết để phân chia một nút bên trong.
- **Lá mẫu tối thiểu:** Số lượng mẫu tối thiểu cần có trên một lá nút.
- **Bootstrap:** Có sử dụng lấy mẫu bootstrap hay không.

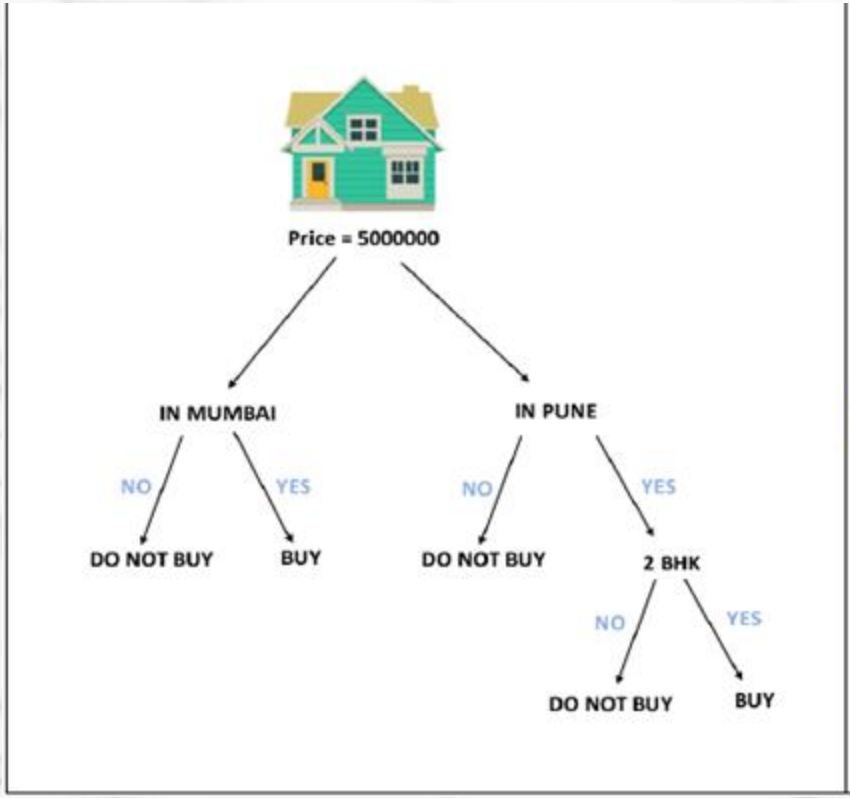
Thực hành:

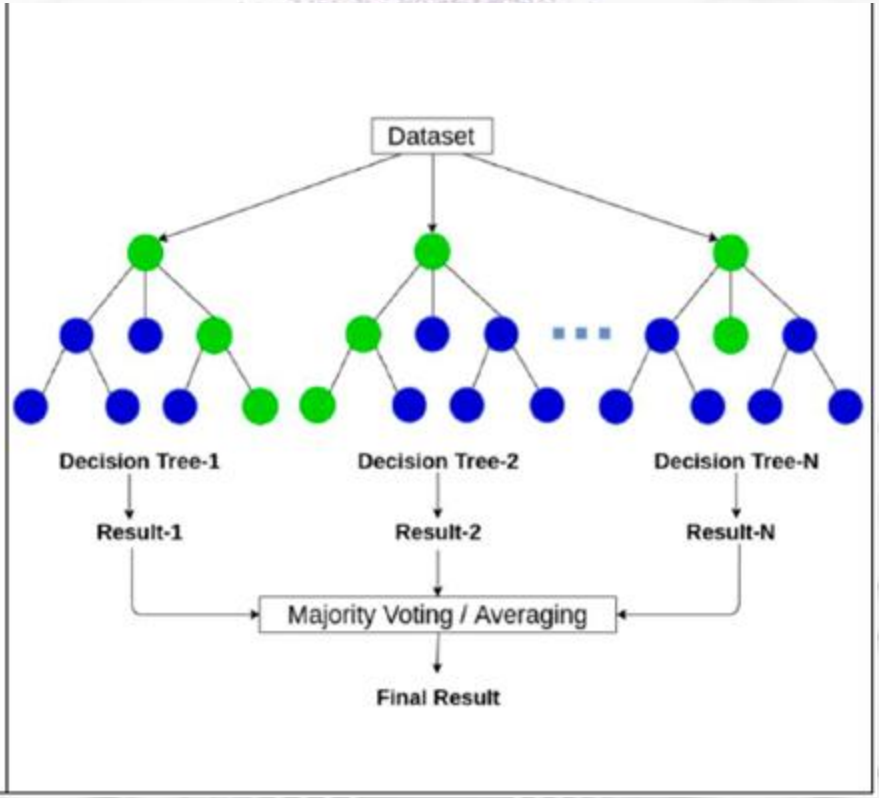
→https://www.datacamp.com/tutorial/random-forests-classifier-python



Sự khác biệt giữa Rừng ngẫu nhiên và Cây quyết định

Feature	Decision Tree	Random Forest
Structure	Single tree	Ensemble of trees
Overfitting	Prone to overfitting	Less prone to overfitting
Accuracy	Generally less accurate	Generally more accurate
Interpretability	Highly interpretable	Less interpretable
Computational cost	Relatively low	Can be computationally expensive





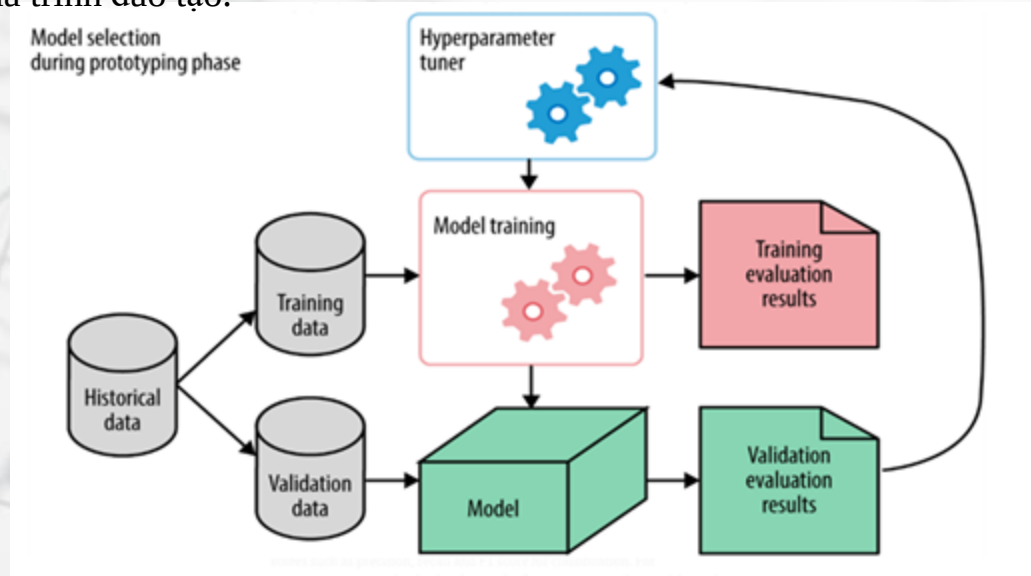
Điều chỉnh siêu tham số

Siêu tham số là các cài đặt không được học từ dữ liệu nhưng được đặt trước khi đào tạo mô hình học máy. Họ kiểm soát hành vi của thuật toán học tập. Việc điều chỉnh các siêu tham số này có thể tác động đáng kể đến hiệu suất của mô hình.

Siêu tham số phổ biến:

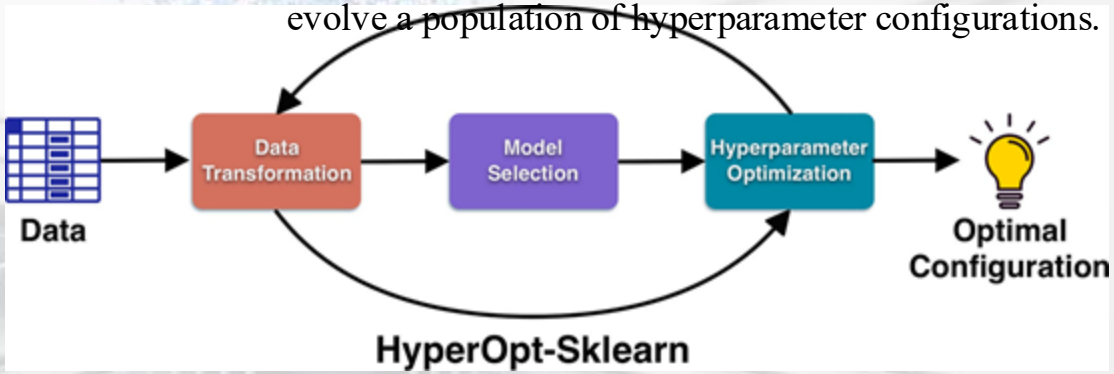
- **Tốc độ học:** Kiểm soát tốc độ mô hình điều chỉnh các tham số của nó trong quá trình đào tạo.
- **Cường độ chính quy hóa:** Kiểm soát mức độ chính quy hóa được áp dụng cho mô hình (ví dụ: chính quy hóa L1 hoặc L2).
- **Số lượng lớp ẩn và nơ-ron:** Xác định độ phức tạp của mạng nơ-ron.

- **Kích thước lô:** Số lượng mẫu được sử dụng trong mỗi lần lặp đào tạo.
- **Kỷ nguyên:** Số lần toàn bộ tập dữ liệu được chuyển qua mô hình trong quá trình đào tạo.



Kỹ thuật điều chỉnh siêu tham số:

1. **Tìm kiếm lưới:**
   - Xác định một lưới các giá trị siêu tham số và huấn luyện một mô hình cho mỗi kết hợp.
   - Tốn thời gian cho không gian tìm kiếm lớn.
2. **Tìm kiếm ngẫu nhiên:**
   - Lấy mẫu ngẫu nhiên các giá trị siêu tham số từ một phân phối được chỉ định.
   - Thường hiệu quả hơn tìm kiếm dạng lưới đối với không gian tìm kiếm lớn.
3. **Tối ưu hóa Bayes:**
   - Sử dụng mô hình xác suất để xây dựng hàm thay thế của hàm mục tiêu.
   - Khám phá không gian tìm kiếm thông minh hơn bằng cách tập trung vào các khu vực đầy hứa hẹn.
4. **Thuật toán tiến hóa:**
   - Lấy cảm hứng từ sự tiến hóa sinh học, các thuật toán này sử dụng các khái niệm như chọn lọc, lai ghép và đột biến để evolve a population of hyperparameter configurations.



Kỹ thuật điều chỉnh siêu tham số

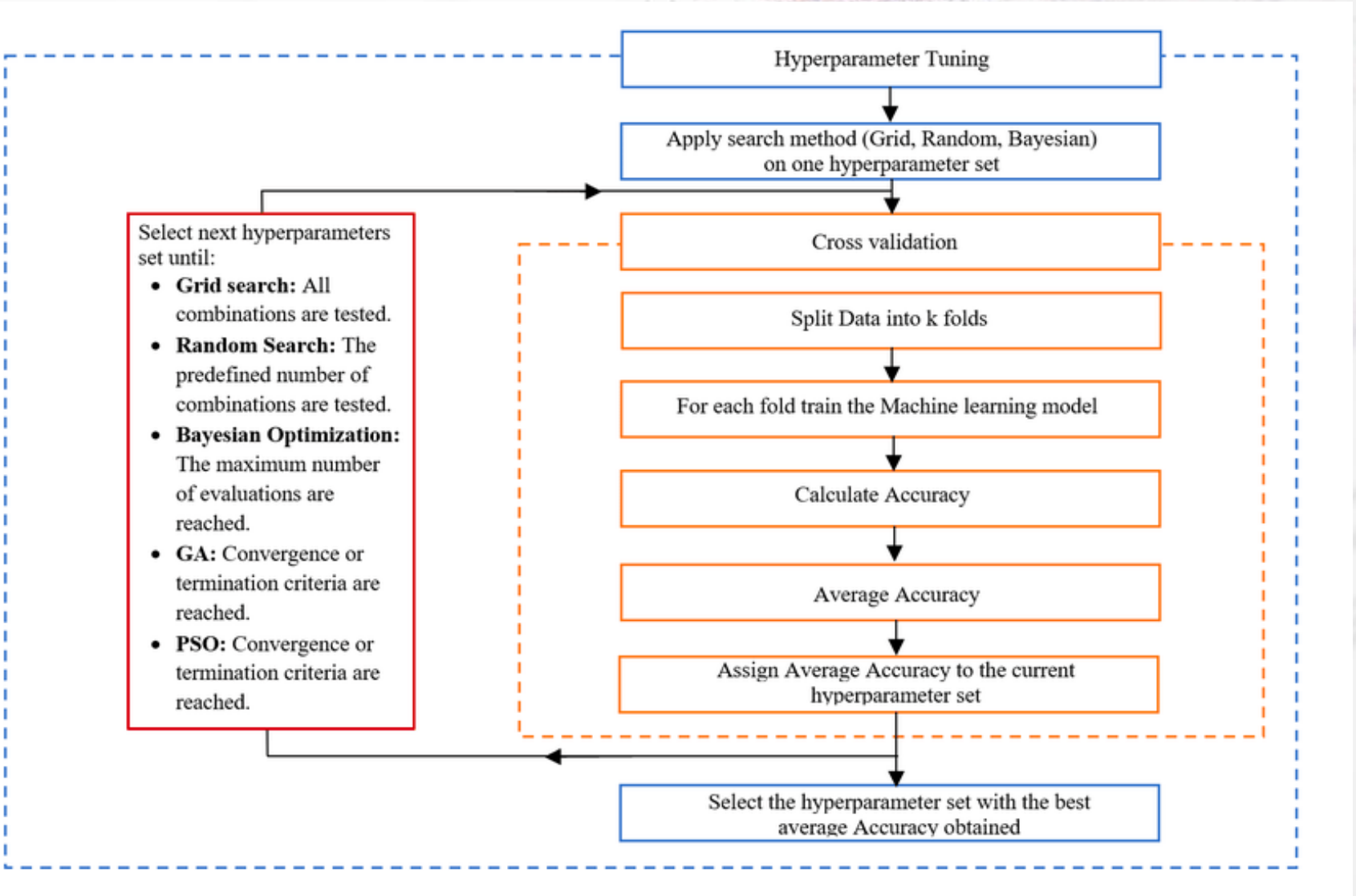
Kỹ thuật điều chỉnh siêu tham số với Scikit-learn

1. Tìm kiếm dạng lưới
2. Tìm kiếm ngẫu nhiên
3. Tối ưu hóa Bayes 4. Thuật toán tiến hóa

Tham khảo:

1) https://scikit-learn.org/1.5/model_selection.html

2) https://www.geeksforgeeks.org/sklearn-model-hyper-điều chỉnh tham số/



Hàm mục tiêu trong hồi quy

Hàm mục tiêu trong hồi quy là một biểu thức toán học định lượng "lỗi" hoặc "khoảng cách" giữa các giá trị dự đoán và giá trị thực tế. Nó phục vụ như một mục tiêu mà mô hình hồi quy nhằm mục đích giảm thiểu.

Việc lựa chọn hàm mục tiêu phụ thuộc vào đặc điểm cụ thể của bài toán hồi quy và các thuộc tính mong muốn của mô hình. Hãy xem xét các yếu tố sau:

-

- **Mức độ nhạy cảm với các ngoại lệ:** Nếu dữ liệu của bạn chứa các ngoại lệ, MAE hoặc Huber Loss có thể phù hợp hơn MSE.
- **Khả năng diễn giải:** RMSE thường được ưu tiên hơn về khả năng diễn giải vì nó có cùng đơn vị với biến mục tiêu.
- **Tối ưu hóa:** MSE thường dễ tối ưu hóa hơn do tính khác biệt của nó.

Chức năng mục tiêu chung:

1. Lỗi bình phương trung bình (MSE):

- Tính toán chênh lệch bình phương trung bình giữa giá trị dự đoán và giá trị thực tế.
- Công thức: $MSE=1/n *\Sigma(yi - \hat{y}i)^2$
- Ưu điểm: Dễ tính toán, vi phân và được sử dụng rộng rãi. -

Nhược điểm: Nhạy cảm với các ngoại lệ do bình phương.

2. Lỗi tuyệt đối trung bình (MAE):

- Tính toán sự khác biệt tuyệt đối trung bình giữa giá trị dự đoán và thực tế.
- Công thức: $MAE=1/n *\Sigma|yi - \hat{y}i|$ Ưu điểm: Ít nhạy cảm với các ngoại lệ hơn MSE.
- Nhược điểm: Không khả vi ở mức 0, khiến việc tối ưu hóa trở nên khó khăn hơn.

3. Lỗi bình phương trung bình gốc (RMSE):

- Căn bậc hai của MSE.
- Công thức: $RMSE= \sqrt{(1/n * \Sigma(yi - \hat{y}i)^2)}$
- Ưu điểm: Có thể giải thích theo cùng đơn vị với biến mục tiêu. -

Nhược điểm: Tương tự như MSE về độ nhạy cảm với các ngoại lệ.

4. Mất Huber:

- Kết hợp các ưu điểm của MSE và MAE bằng cách sử dụng tổn thất bậc hai cho các lỗi nhỏ và tổn thất tuyến tính cho các lỗi lớn. -Công thức:

  Mất Huber = $1/n * \Sigma(\delta^2 * (yi - \hat{y}i)^2 / 2$, nếu $|yi - \hat{y}i| \leq \delta$, Mất Huber = $|yi - \hat{y}i| - \delta^2/2$, ngược lại
- Ưu điểm: Mạnh mẽ đối với các ngoại lệ trong khi vẫn duy trì khả năng khác biệt.

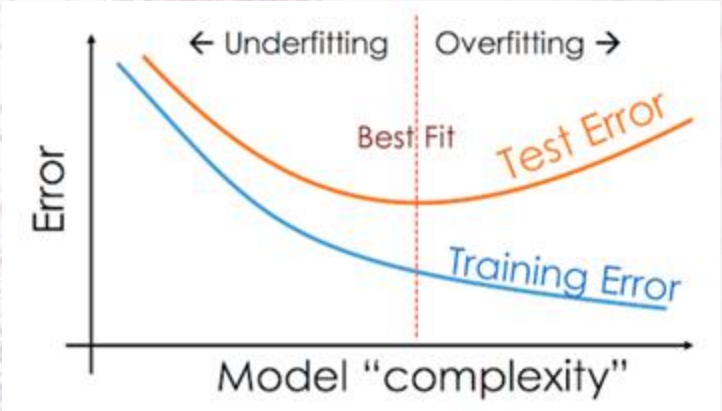
Trang bị quá mức và chưa trang bị đầy đủ trong học máy

Trang bị quá mức

Quá khớp xảy ra khi một mô hình học dữ liệu huấn luyện quá tốt, thu được các mẫu nhiễu và không liên quan. Điều này dẫn đến một mô hình hoạt động cực kỳ tốt trên dữ liệu huấn luyện nhưng lại hoạt động kém trên dữ liệu mới, chưa được nhìn thấy.

Đặc điểm của trang bị quá mức:

- Hiệu suất cao trên dữ liệu huấn luyện: Mô hình đạt được độ chính xác rất cao hoặc sai số thấp trên tập huấn luyện.
- Hiệu suất kém trên dữ liệu xác thực/kiểm tra: Hiệu suất của mô hình giảm đáng kể khi được đánh giá trên dữ liệu không nhìn thấy được.
- Mô hình phức tạp: Mô hình có thể có quá nhiều tham số hoặc tính năng, khiến mô hình có xu hướng ghi nhớ dữ liệu huấn luyện thay vì học các mẫu cơ bản.

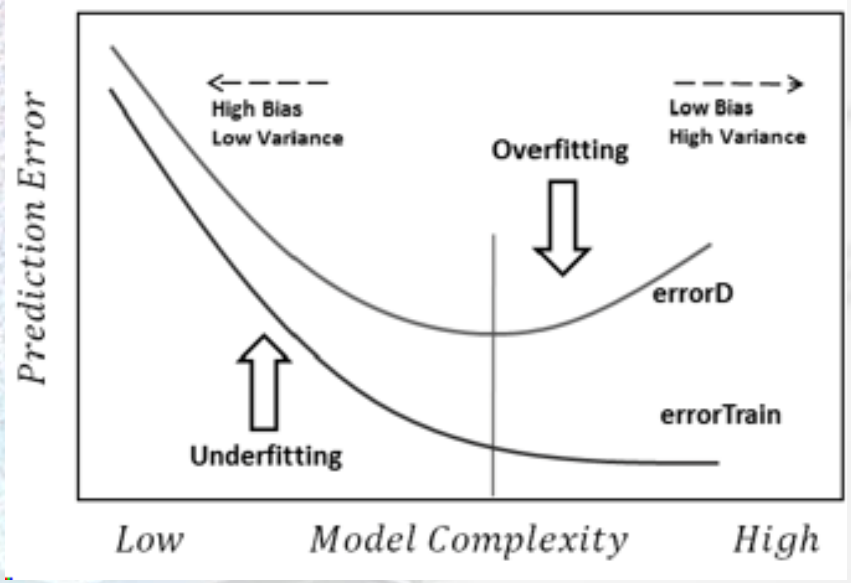


Không phù hợp

Việc trang bị chưa đầy đủ xảy ra khi một mô hình quá đơn giản để nắm bắt được các mẫu cơ bản trong dữ liệu. Điều này dẫn đến một mô hình hoạt động kém trên cả dữ liệu huấn luyện và xác thực/kiểm tra.

Đặc điểm của việc thiếu trang bị:

- Hiệu suất kém trên cả dữ liệu huấn luyện và xác nhận/kiểm tra: Mô hình luôn đạt được độ chính xác thấp hoặc sai số cao trên cả hai bộ.
- Mô hình đơn giản: Mô hình có thể có quá ít tham số hoặc tính năng, hạn chế khả năng tìm hiểu các mối quan hệ phức tạp.



Giải quyết vấn đề trang bị quá mức và không phù hợp

Giải quyết vấn đề trang bị quá mức và không phù hợp

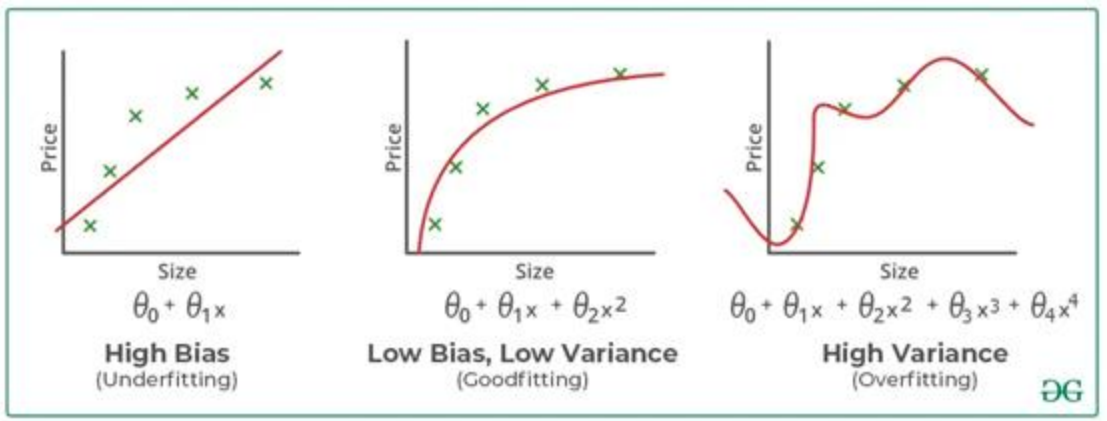
1. Chính quy hóa:

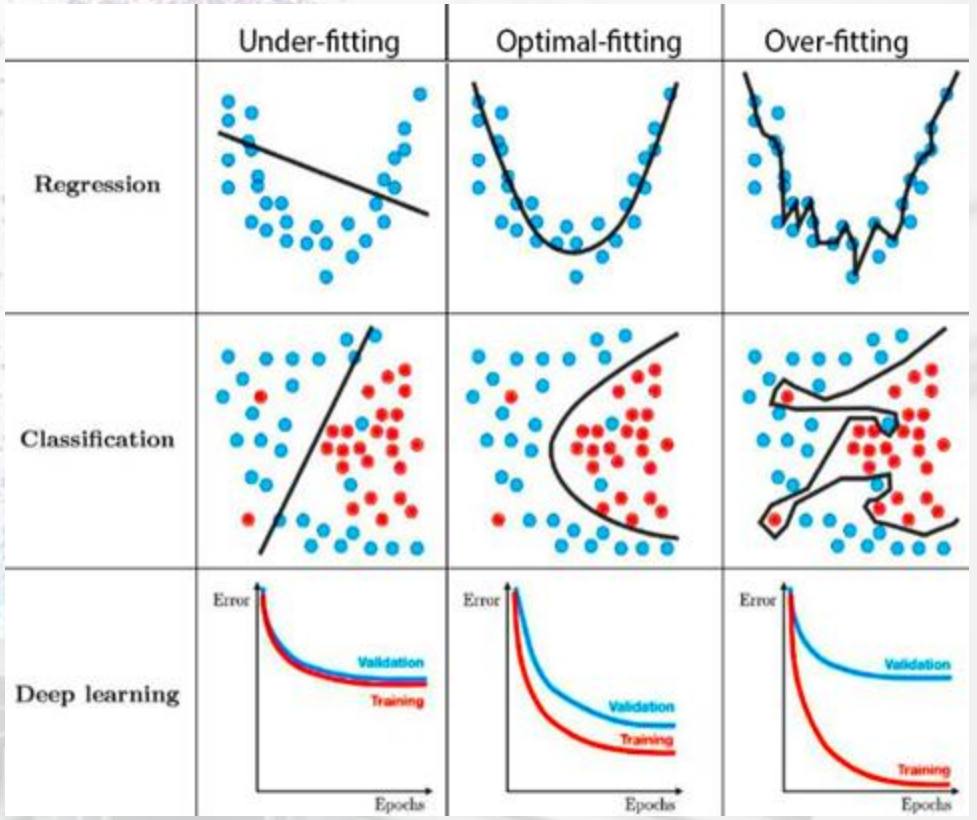
- **Chính quy hóa L1 (Lasso):** Đưa ra một thuật ngữ phạt khuyến khích tính thưa thớt, nghĩa là nhiều tham số mô hình trở thành số 0. Điều này có thể giúp ngăn chặn việc trang bị quá mức bằng cách giảm độ phức tạp của mô hình. - **Chính quy hóa L2 (Ridge):** Thêm thời hạn phạt nhằm ngăn cản các giá trị tham số lớn, ngăn mô hình trở nên quá nhạy cảm với những thay đổi nhỏ trong dữ liệu đầu vào.

1. **Xác thực chéo:** Chia dữ liệu thành nhiều phần và huấn luyện mô hình trên các tập hợp con khác nhau để đánh giá hiệu suất của nó trên dữ liệu không nhìn thấy. Điều này giúp sớm xác định quá khớp hoặc thiếu khớp trong quá trình lập mô hình. 2. **Dừng sớm:** Giám sát hiệu suất của mô hình trên một tập hợp xác thực trong quá trình đào tạo và dừng quá trình đào tạo khi hiệu suất bắt đầu suy giảm, ngăn ngừa việc trang bị quá mức.

3. **Kỹ thuật tính năng:** Việc tạo các tính năng mới hoặc chuyển đổi các tính năng hiện có có thể cải thiện khả năng của mô hình trong việc nắm bắt các mẫu có liên quan và giảm việc trang bị quá mức.

4. **Mô hình đơn giản hơn:** Nếu mô hình quá khớp, hãy cân nhắc sử dụng mô hình đơn giản hơn với ít tham số hơn.



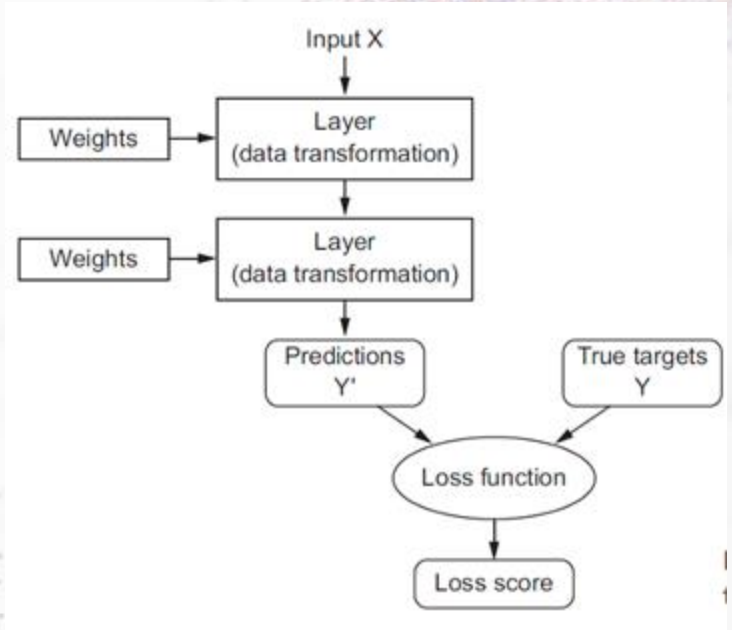


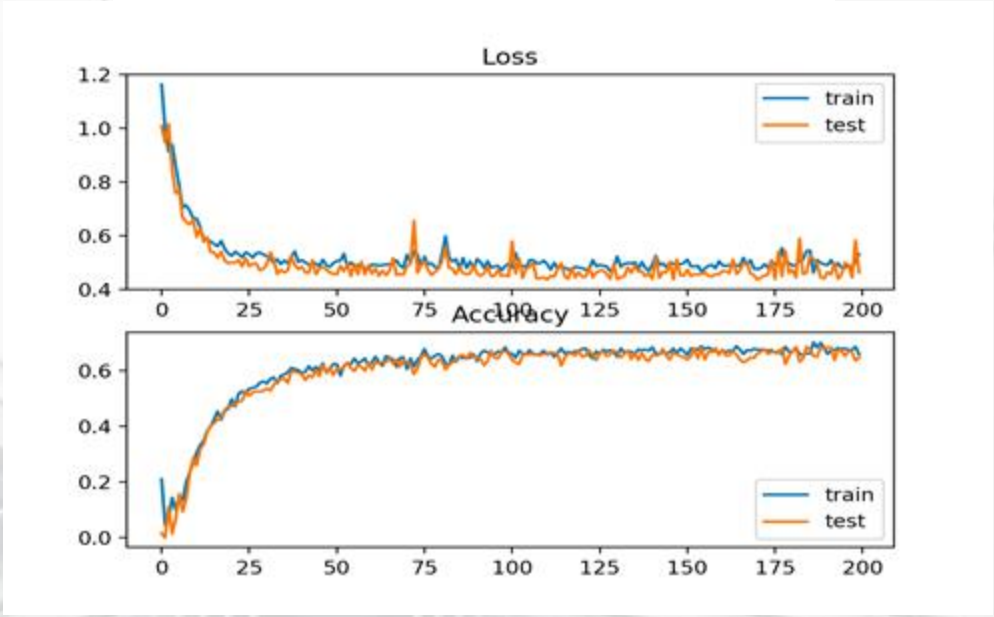
Hàm mất mát trong học máy

A**hàm mất mát**, còn được gọi là hàm chi phí hoặc hàm mục tiêu, là hàm toán học định lượng "lỗi" hoặc "khoảng cách" giữa giá trị dự đoán và giá trị thực tế trong mô hình học máy. Nó đóng vai trò là mục tiêu mà mô hình hướng tới giảm thiểu trong quá trình đào tạo.

Các hàm mất mát thường được sử dụng trong học máy

Task	Error type	Loss function	Note
Regression	Mean-squared error	$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Easy to learn but sensitive to outliers (MSE, L2 loss)
	Mean absolute error	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Robust to outliers but not differentiable (MAE, L1 loss)
Classification	Cross entropy = Log loss	$-\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] =$	Quantify the difference between two probability





Chọn hàm mất đúng:

Việc lựa chọn hàm mất mát phụ thuộc vào đặc điểm cụ thể của bài toán và các thuộc tính mong muốn của mô hình. Hãy xem xét các yếu tố sau:

- Loại vấn đề: Hồi quy hay phân loại?
- Độ nhạy cảm với các ngoại lệ: Dữ liệu có bị nhiễu hoặc có xu hướng xuất hiện các ngoại lệ không?
- Khả năng diễn giải: Bạn có cần sai số có cùng đơn vị với biến mục tiêu không?

Giảm thiểu tổn thất thường xuyên

Giảm thiểu tổn thất thường xuyên là một kỹ thuật được sử dụng trong học máy để ngăn chặn việc trang bị quá mức bằng cách thêm một số hạng phạt vào hàm mất mát. Thời hạn phạt này được thiết kế để ngăn cản các mô hình quá phức tạp, dễ bị trang bị quá mức.

Hàm mất

Hàm mất mát định lượng "lỗi" hoặc "khoảng cách" giữa các giá trị dự đoán và giá trị thực tế. Các hàm mất mát phổ biến bao gồm:

- Lỗi bình phương trung bình (MSE)
- Sai số tuyệt đối trung bình (MAE)
- Mất Entropy chéo

Thuật ngữ chính quy

Thuật ngữ chính quy được thêm vào hàm mất để xử phạt các giá trị tham số lớn. Hai loại chính quy hóa phổ biến nhất là:

1. **Chính quy hóa L1 (Lasso):**
   - Thêm giá trị tuyệt đối của các tham số vào hàm mất mát.
   - Khuyến khích tính thưa thớt, nghĩa là nhiều tham số trở thành số 0.
   - Có thể hữu ích cho việc lựa chọn tính năng.
2. **Chính quy hóa L2 (Ridge):**
   - Thêm bình phương của các tham số vào hàm mất.
   - Không khuyến khích các giá trị tham số lớn.
   - Có thể giúp ngăn chặn việc trang bị quá mức bằng cách giảm phương sai của mô hình.

Mục tiêu tổng thể. Mục tiêu của việc giảm thiểu tổn thất thường xuyên là giảm thiểu hàm mục tiêu sau:

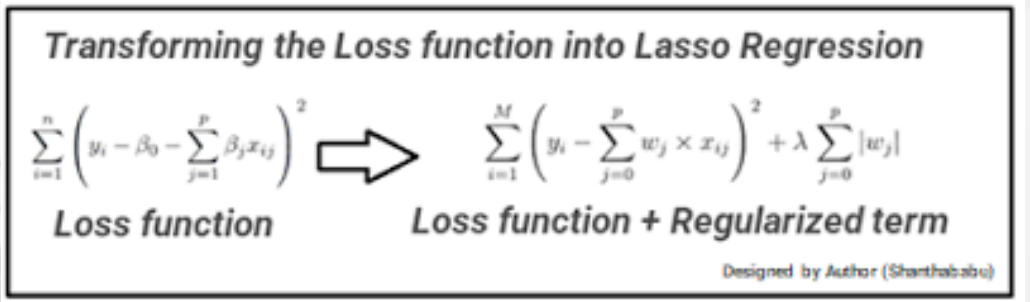
$$\text{Mất}(w) + \lambda * R(w)$$

ở đâu:

- Mất(w) là hàm mất.
- λlà tham số chính quy hóa, kiểm soát cường độ của chính quy hóa.
- R(w) là thuật ngữ chính quy hóa (L1 hoặc L2).

Lợi ích của việc giảm thiểu tổn thất thường xuyên

- **Ngăn chặn việc trang bị quá mức:** Bằng cách xử phạt các giá trị tham số lớn, việc chính quy hóa có thể giúp ngăn các mô hình trở nên quá phức tạp và ghi nhớ dữ liệu huấn luyện.
- **Cải thiện khả năng khái quát hóa:** Các mô hình chính quy có xu hướng khái quát hóa tốt hơn đối với dữ liệu không nhìn thấy được.
- **Lựa chọn tính năng:** Chính quy L1 có thể được sử dụng để lựa chọn tính năng bằng cách đặt nhiều tham số về 0.



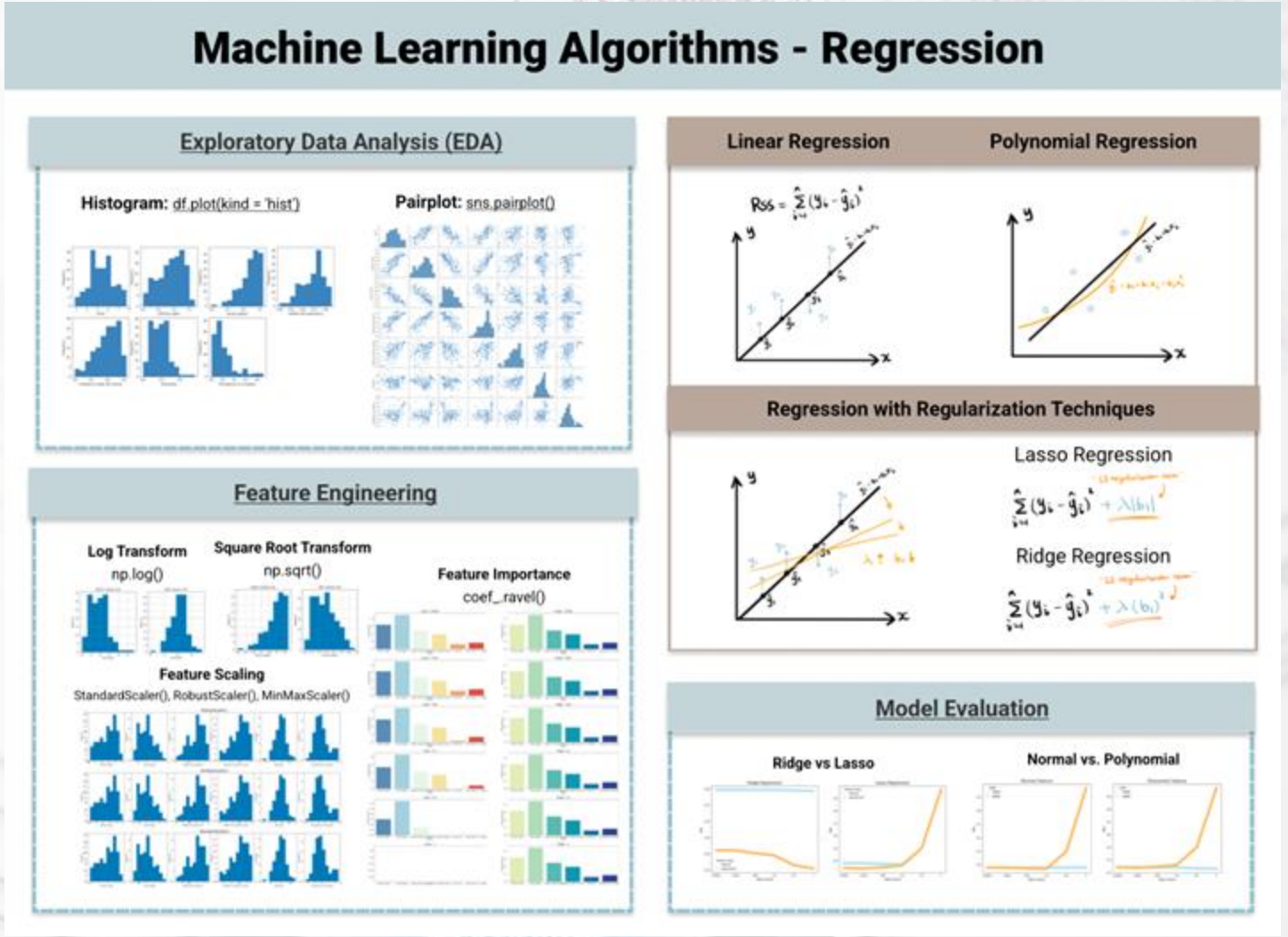
Học máy cho hồi quy

Regression is a statistical method used to model the relationship between a dependent variable (the outcome) and one or more independent variables (predictors). Trong học máy, thuật toán hồi quy được sử dụng để dự đoán các giá trị số liên tục.

Regression evaluation metrics are used to evaluate the performance of regression models, such as MSE,RMSE, MAE, MAPE,... They quantify how well a model's predictions align with the actual values.

Ứng dụng hồi quy

- Dự đoán doanh số bán hàng: Dự báo doanh số bán hàng trong tương lai dựa trên dữ liệu lịch sử.
- Dự đoán giá cổ phiếu: Dự đoán giá cổ phiếu trong tương lai. - Dự báo nhu cầu: Dự đoán nhu cầu về sản phẩm hoặc dịch vụ.
- Dự đoán giá bất động sản: Ước tính giá của một bất động sản dựa trên đặc điểm của nó.
- Dự đoán Giá trị trọn đời của Khách hàng: Dự đoán tổng doanh thu mà một khách hàng sẽ tạo ra trong suốt vòng đời của họ.



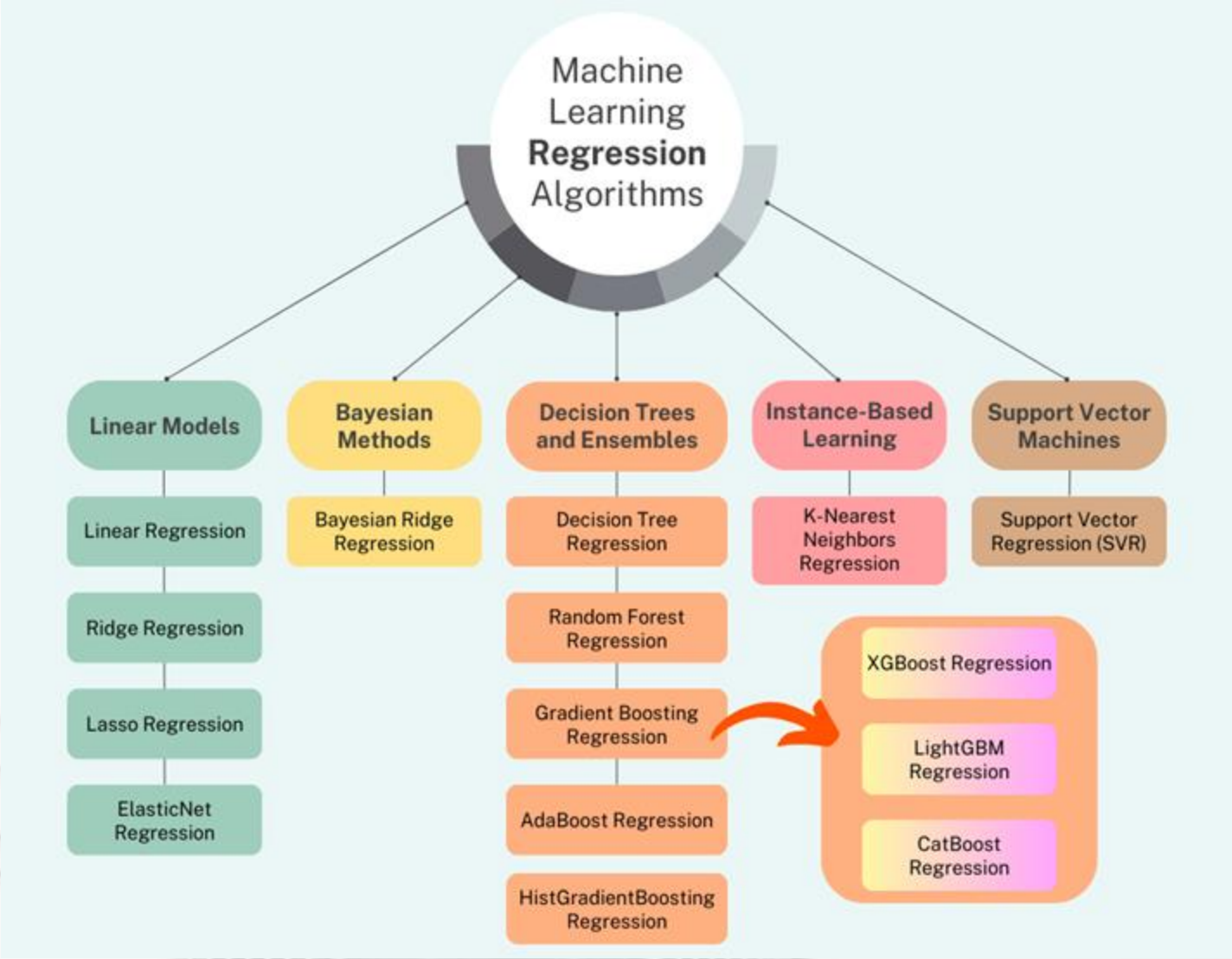
Các chỉ số đánh giá hồi quy

Lỗi bình phương trung bình (MSE)	**MSE = 1/n * Σ(yi - ŷi)²** Đo lường bình phương trung bình của sự khác biệt giữa dự đoán và những giá trị thực tế. MSE thấp hơn thường cho thấy hiệu suất tốt hơn.
Lỗi bình phương trung bình gốc (RMSE)	**RMSE = √(MSE)** Hình vuông gốc của MSE, thường được ưa thích bởi vì nó đang ở trong cùng đơn vị làm biến mục tiêu.
Lỗi tuyệt đối trung bình (MAE)	**MAE = 1/n * Σ|yi - ŷi|** Đo lường trung bình tuyệt đối chênh lệch giữa giá trị dự đoán và giá trị thực tế. Ít nhạy cảm hơn đến các ngoại lệ hơn MSE.
R bình phương (R²)	**R² = 1 - SSres / SStot** Đo tỷ lệ phương sai của biến phụ thuộc được giải thích bởi các biến độc lập. Cao hơn R² biểu thị sự phù hợp tốt hơn.
R bình phương đã điều chỉnh	**R² đã điều chỉnh = 1 - (1 - R²) * (n - 1) / (n -p - 1)** Phạt R² vì thêm các yếu tố dự đoán không cần thiết.
Lỗi phần trăm tuyệt đối trung bình (MAPE)	**BẢN ĐỒ = 100 * 1/n * Σ|yi - ŷi| / |yi|** Đo lường sai số phần trăm trung bình giữa giá trị dự đoán và giá trị thực tế. Hữu ích cho việc so sánh các mô hình hồi quy mô khác nhau.
Trung bình có trọng số Lỗi bình phương (WMSE)	**WMSE = 1/n * Σwi * (yi - ŷi)2** Gán các trọng số khác nhau cho các lỗi dựa trên tầm quan trọng của các điểm dữ liệu.

Thuật toán hồi quy học máy

Thuật toán hồi quy phổ biến

- **Hồi quy tuyến tính:** Một mô hình đơn giản giả định mối quan hệ tuyến tính giữa các biến phụ thuộc và biến độc lập.
- **Hồi quy đa thức:** Một mô hình linh hoạt hơn có thể nắm bắt các mối quan hệ phi tuyến tính bằng cách khớp đường cong đa thức với dữ liệu.
- **Hồi quy sườn:** Một kỹ thuật chính quy hóa bổ sung một số hạng phạt vào hàm mất mát để ngăn chặn quá khớp.
- **Hồi quy Lasso:** Một kỹ thuật chính quy hóa khác sử dụng thuật ngữ phạt khác để khuyến khích sự thưa thớt trong mô hình, nghĩa là nó có thể chọn các tính năng quan trọng nhất.
- **Lưới đàn hồi:** Sự kết hợp giữa hồi quy Ridge và Lasso, mang lại lợi ích cho cả hai.
- **Cây quyết định:** Có thể được sử dụng để hồi quy bằng cách đưa ra dự đoán dựa trên giá trị trung bình của biến mục tiêu trong các nút lá.
- **Random Forest:** Một phương pháp tổng hợp kết hợp nhiều cây quyết định để cải thiện độ chính xác và giảm tình trạng trang bị quá mức.
- **Máy vectơ hỗ trợ (SVM):** Có thể được sử dụng để hồi quy bằng cách tìm một siêu phẳng giúp tối đa hóa lề giữa các điểm dữ liệu.
- **Mạng thần kinh:** Các mô hình học sâu có thể tìm hiểu các mối quan hệ phi tuyến tính phức tạp giữa các tính năng và biến mục tiêu.



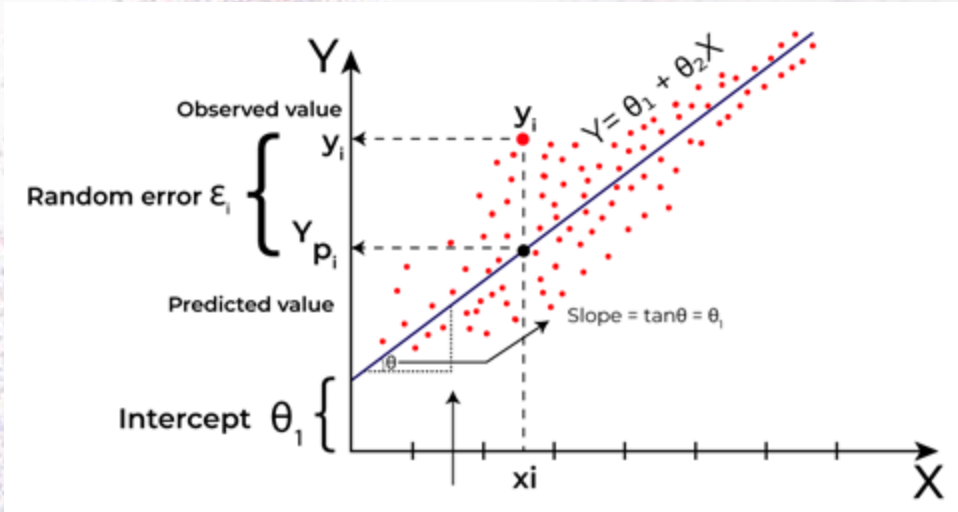
Hồi quy tuyến tính: Nền tảng về học máy

Hồi quy tuyến tính là một phương pháp thống kê được sử dụng để mô hình hóa mối quan hệ giữa một biến phụ thuộc (kết quả) và một hoặc nhiều biến độc lập (các yếu tố dự đoán). Nó giả định mối quan hệ tuyến tính giữa các biến và nhằm mục đích tìm ra đường phù hợp nhất để thể hiện mối quan hệ này.

Phương trình của mô hình hồi quy tuyến tính là: $y = mx + b$

ở đâu:

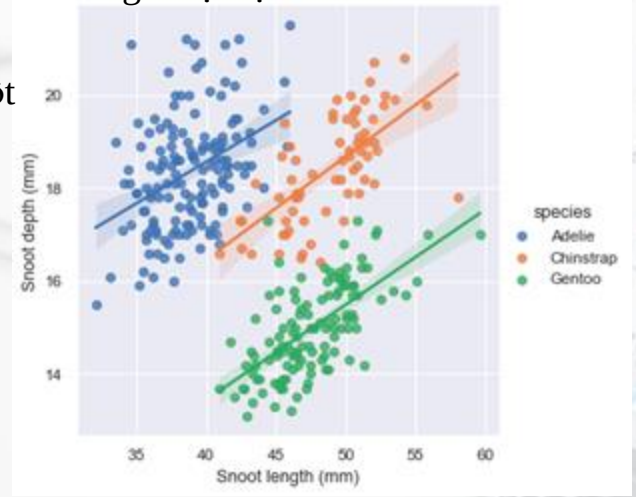
- y là biến phụ thuộc (kết quả)
- x là biến độc lập (dự báo)
- m là độ dốc của đường
- b là điểm chặn y

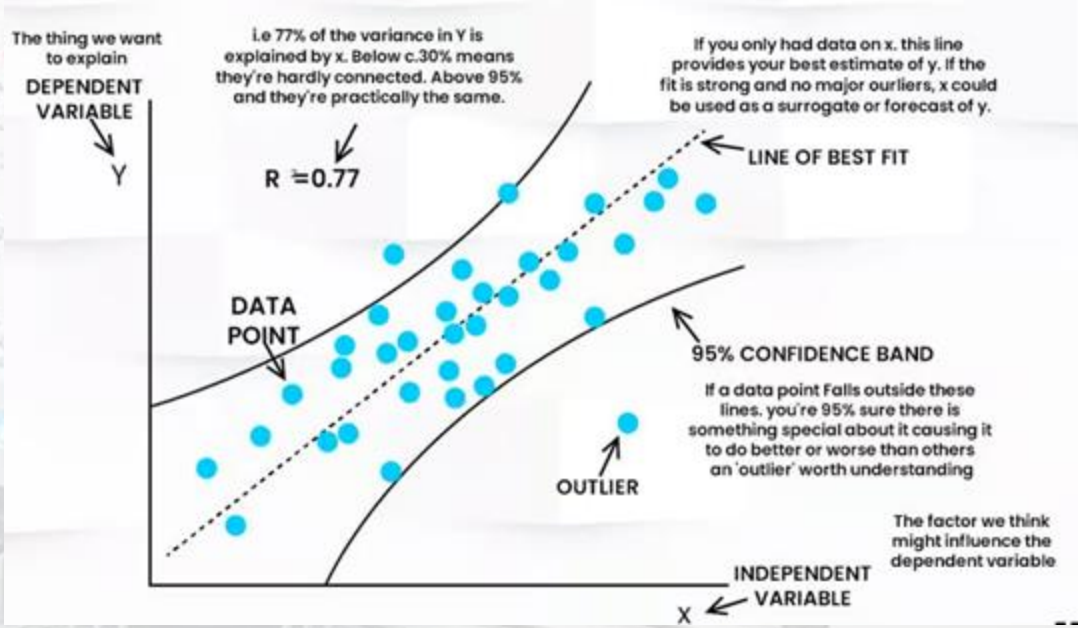


Tìm đường dây phù hợp nhất. Mục tiêu của hồi quy tuyến tính là tìm các giá trị m và b sao cho sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất. Điều này thường được thực hiện bằng cách sử dụng phương pháp bình phương tối thiểu, phương pháp này tính toán đường cực tiểu hóa tổng các chênh lệch bình phương giữa giá trị dự đoán và giá trị thực tế.

Các loại hồi quy tuyến tính:

- Hồi quy tuyến tính đơn giản: Liên quan đến một biến độc lập duy nhất.
  - Phương trình: **y = mx + b**
  - Ví dụ: Dự đoán giá nhà dựa trên kích thước của ngôi nhà.
- Hồi quy tuyến tính bội: Liên quan đến nhiều các biến độc lập.
  - Phương trình: **y = b0 + b1x1 + b2x2 + ... + bn*xn**
  - Ví dụ: Dự đoán giá xe dựa trên các yếu tố như quãng đường đã đi, tuổi đời, nhãn hiệu và tính năng.





Bình phương tối thiểu thông thường (OLS)

Bình phương tối thiểu thông thường (OLS) là phương pháp hồi quy tuyến tính phổ biến và đơn giản nhất. Nó nhằm mục đích tìm ra đường phù hợp nhất để giảm thiểu tổng bình phương chênh lệch giữa giá trị dự đoán và giá trị thực tế.

Trong OLS, mục tiêu là giảm thiểu tổng bình phương phần dư (SSR):

$$\mathbf{SSR = \Sigma(y_i - \hat{y}_i)^2}$$

ở đâu: yi - th

e giá trị quan sát của biến phụ thuộc.

ŷi - giá trị dự đoán của biến phụ thuộc.

Ước tính OLS của các hệ số hồi quy (độ dốc và giao điểm) có được bằng cách giải các phương trình chuẩn sau:

$$\min_w ||Xw - y||_2^2$$

Ở đâu, giá trị dự đoán:

$$\hat{y}(w, x) = w_0 + w_1 x_1 + \ldots + w_p x_p$$

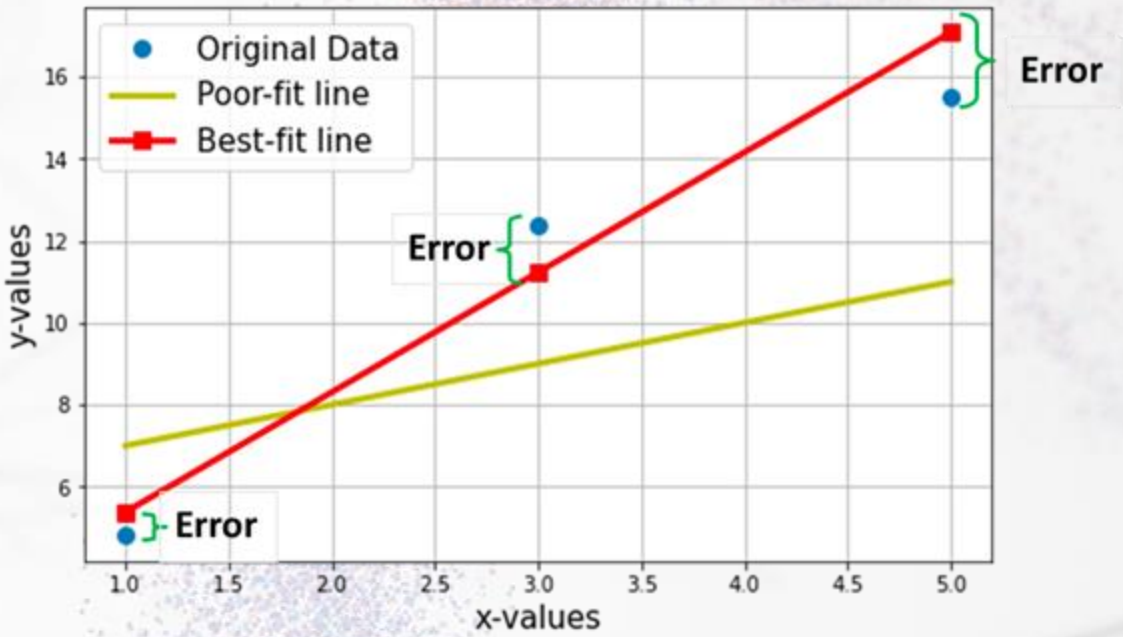
vectơ

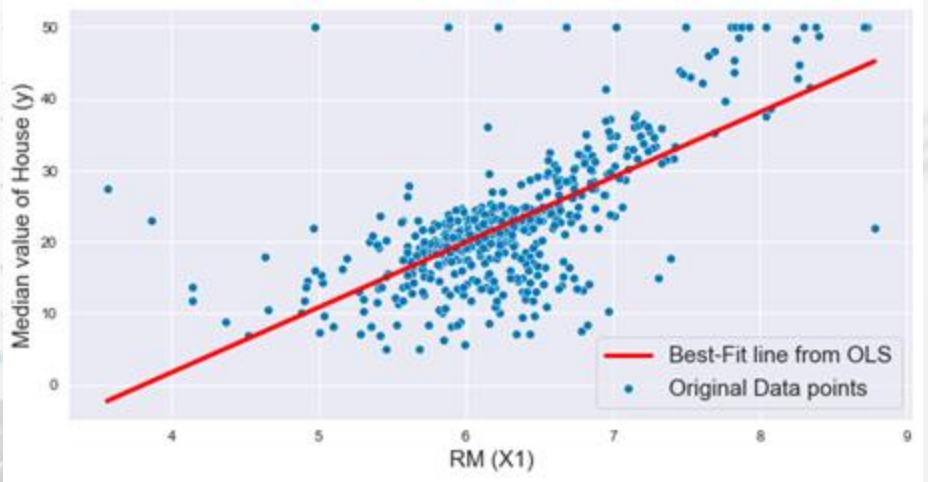
$w = (w_1, \ldots, w_p)$ as `coef_` and w_0 as `intercept_`.

Sử dụng scikit-learn,

- Hồi quy tuyến tính https://scikit-learn.org/stable/modules/Tuyến_model.html#normal-least-squares - Ví dụ về hồi quy tuyến tính https://scikit-learn.org/stable/auto_examples/Tuyến_model/plot_ols.html#sphx-glr-auto-examples-tuyến tính-model-plot-ols-py

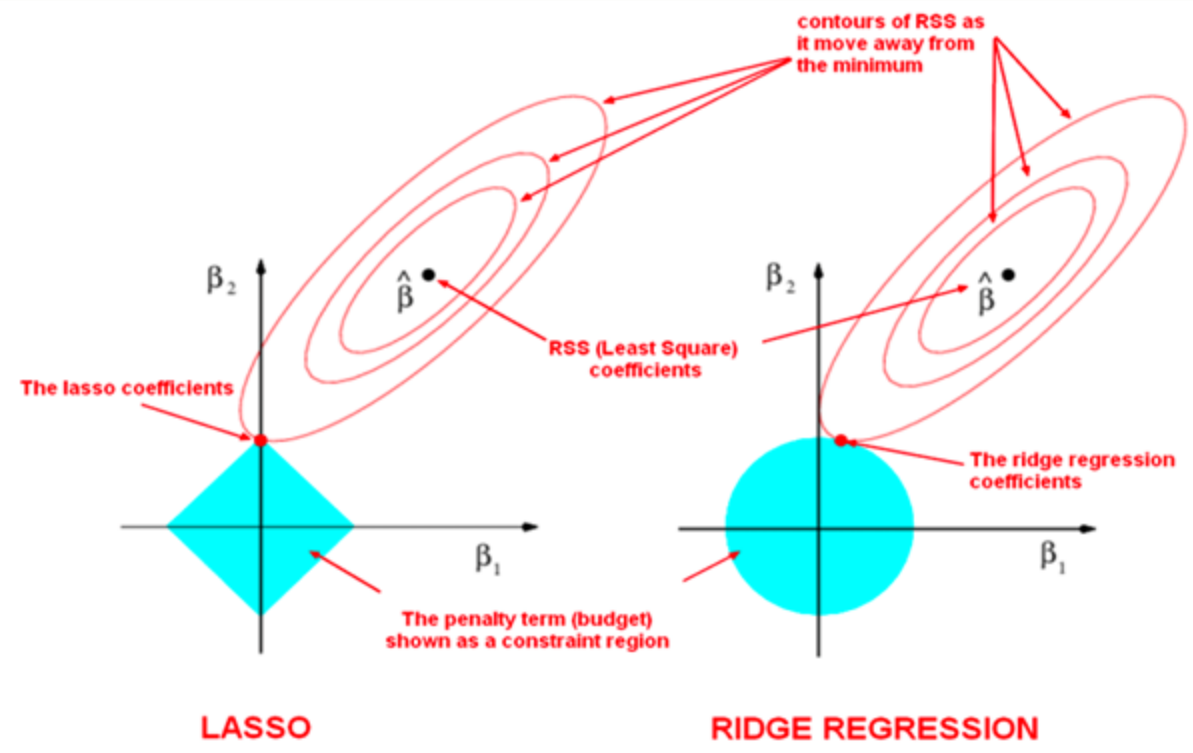
- Hướng dẫn toàn diện về hồi quy OLS https://www.analyticsvidhya.com/blog/2023/01/a-comprehensive-guide-to-ols-Regression-part-1/





Hồi quy Lasso và Hồi quy Ridge

LASSO (Toán tử lựa chọn và co rút tuyệt đối nhỏ nhất) và Hồi quy sườn đều là các kỹ thuật chính quy hóa được sử dụng để ngăn chặn việc trang bị quá mức trong các mô hình hồi quy tuyến tính. Mặc dù chúng có những điểm tương đồng nhưng chúng có những đặc điểm và ứng dụng riêng biệt.



Tính năng	LASSO	Hồi quy sườn
Hình phạt thời hạn	$\Sigma\|wj\|$	$\Sigma(wj^2)$
Lựa chọn tính năng	Yes	No
Hệ số Sự co ngót	Đặt một sốhệ số bằng 0	Thu gọn tất cả các hệ số về 0
Độ lệch-Phương sai Đánh đổi	Độ lệch cao hơn, phương sai thấp hơn	Độ lệch thấp hơn, cao hơn phương sai

Khi nào nên sử dụng cái nào - LASSO:

- ○Khi lựa chọn tính năng là mong muốn.
- ○Khi mô hình thực sự được cho là thưa thớt (có nhiều hệ số bằng 0).

- Hồi quy sườn:

- ○Khi tất cả các tính năng được cho là có liên quan.
- ○Khi việc trang bị quá mức là một mối quan tâm và việc lựa chọn tính năng là không cần thiết.

Phương pháp tiếp cận kết hợp

- **Lưới đàn hồi**: Kết hợp các hình phạt L1 và L2, mang lại sự cân bằng giữa lựa chọn tính năng và độ co rút.
- **LASSO thích ứng**: Điều chỉnh tham số chính quy hóa cho từng tính năng dựa trên ước tính ban đầu của chúng.

Sử dụng scikit-learn,

→Hồi quy Laso

https://scikit-learn.org/1.5/modules/Tuyến_model.html#lasso

→ Hồi quy Rigdge

https://scikit-learn.org/1.5/modules/Tuyến_model.html#regression →

Thực hành

https://www.datacamp.com/tutorial/tutorial-lasso-ridge-regression

Hồi quy đa thức

Hồi quy đa thức là một dạng phân tích hồi quy trong đó mối quan hệ giữa các biến độc lập và biến phụ thuộc được mô hình hóa theo đa thức bậc n.

Đối với Nhiều biến, phép tính ma trận được thực hiện bởi

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^m \\ 1 & x_2 & x_2^2 & \dots & x_2^m \\ 1 & x_3 & x_3^2 & \dots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_n \end{bmatrix},$$

Hàm chi phí của hồi quy đa thức.

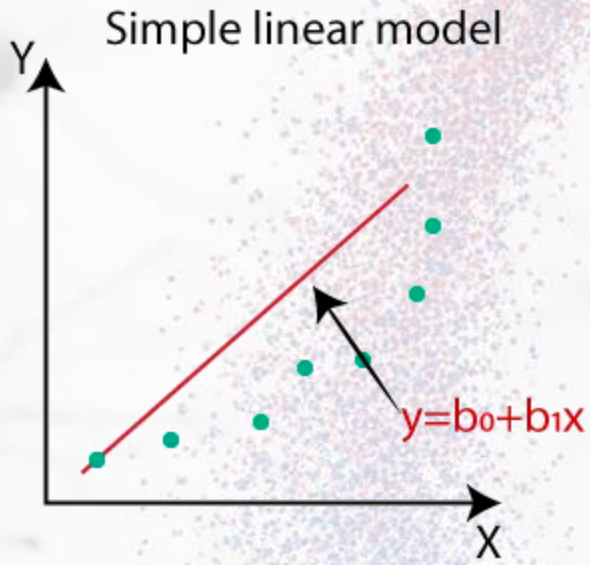
Hàm chi phí đo lường hiệu suất của mô hình bằng cách tính sai số trung bình giữa dự đoán và giá trị thực tế của mô hình (hàm mất mát cho từng điểm dữ liệu). Hãy coi hàm chi phí là "điểm" tổng thể và hàm tổn thất là "điểm" riêng lẻ.

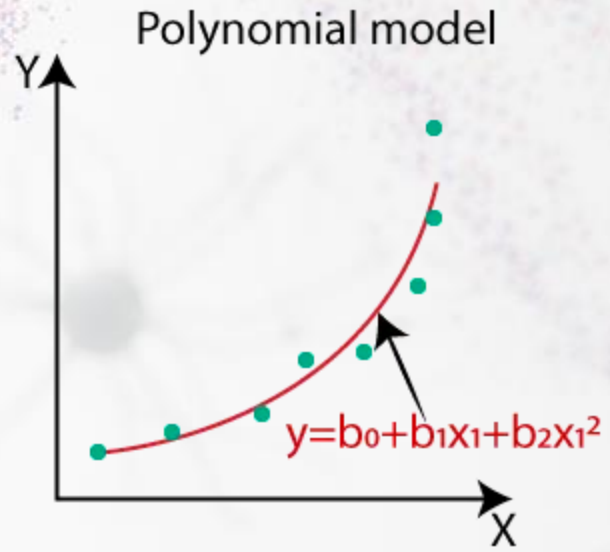
res."

$$J = \frac{1}{n}\sum_{i=1}^{n}(pred_i - y_i)^2$$

Hồi quy đa thức có thể cải thiện sự phù hợp của mô hình bằng cách tạo một đường cong phù hợp hơn với dữ liệu của bạn, có khả năng làm giảm giá trị của hàm chi phí. Các đa thức bậc cao hơn có thể đạt được sự phù hợp chính xác hơn nữa.

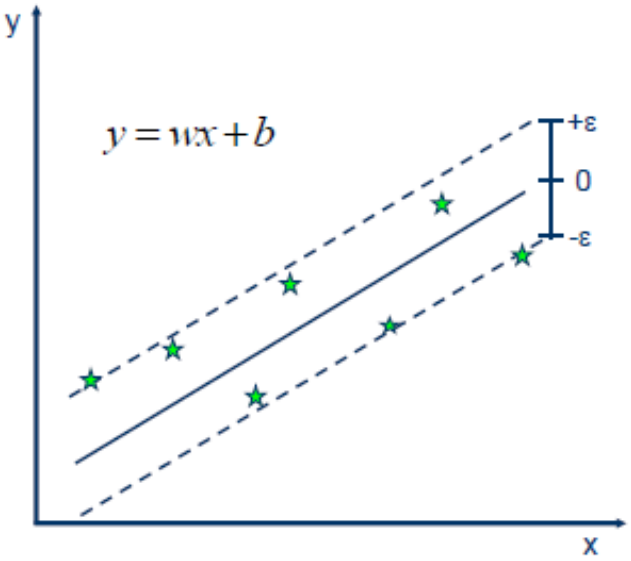
Để giảm thiểu hàm chi phí và cải thiện hiệu suất của mô hình, chúng ta có thể sử dụng phương pháp giảm độ dốc. Thuật toán này điều chỉnh lặp đi lặp lại các trọng số của mô hình để giảm sai số giữa giá trị dự đoán và giá trị thực tế.





Simple Linear Regression	$y = b_0 + b_1 x_1$
Multiple Linear Regression	$y = b_0 + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n$
Polynomial Linear Regression	$y = b_0 + b_1 x_1 + b_2 x_1^2 + \ldots + b_n x_1^n$

SVM cho hồi quy (SVR)

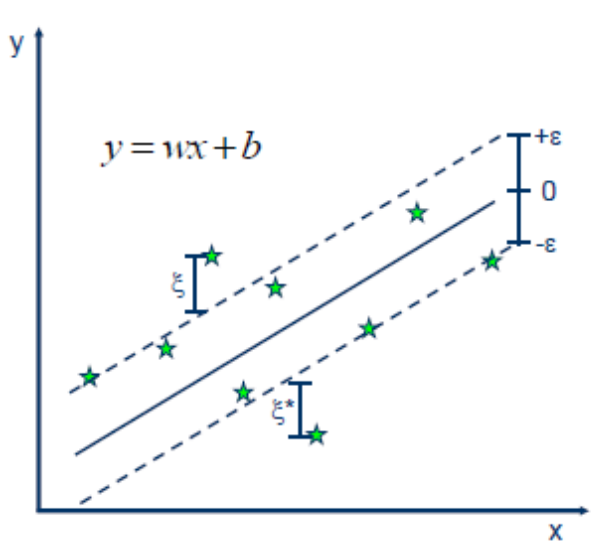


- Solution:

$$\min \frac{1}{2}\|w\|^2$$

- Constraints:

$$y_i - wx_i - b \le \varepsilon$$
$$wx_i + b - y_i \le \varepsilon$$



- Minimize:

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}\left(\xi_i + \xi_i^*\right)$$

- Constraints:

$$y_i - wx_i - b \le \varepsilon + \xi_i$$
$$wx_i + b - y_i \le \varepsilon + \xi_i^*$$
$$\xi_i, \xi_i^* \ge 0$$

Linear SVR

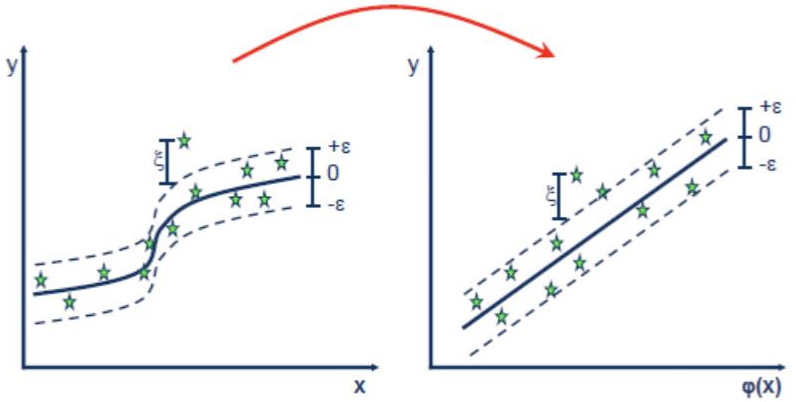
$$y = \sum_{i=1}^{N}\left(\alpha_i - \alpha_i^*\right)\cdot\langle x_i, x\rangle + b$$

Non-linear SVR

The kernel functions transform the data into a higher dimensional feature space to make it possible to perform the linear separation.

$$y = \sum_{i=1}^{N}\left(\alpha_i - \alpha_i^*\right)\cdot\langle\varphi(x_i), \varphi(x)\rangle + b$$

$$y = \sum_{i=1}^{N}\left(\alpha_i - \alpha_i^*\right)\cdot K(x_i, x) + b$$



Kernel functions

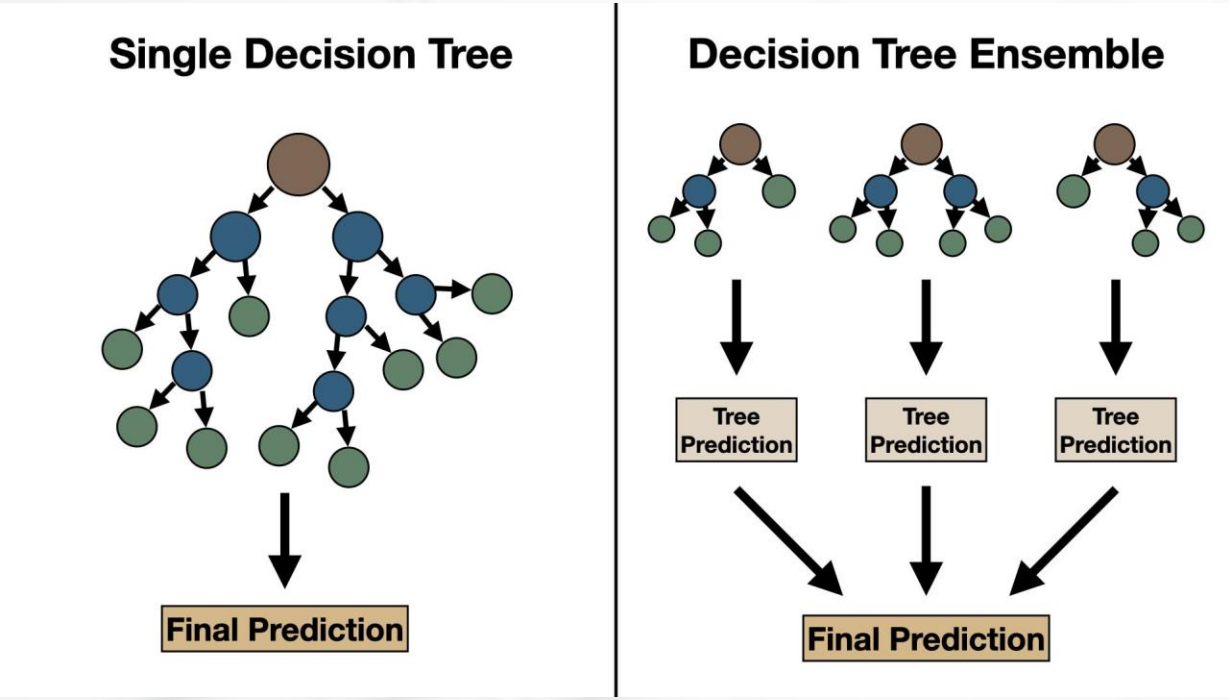
Polynomial

$$k(\mathrm{x}_i, \mathrm{x}_j) = (\mathrm{x}_i.\mathrm{x}_j)^d$$

Gaussian Radial Basis function

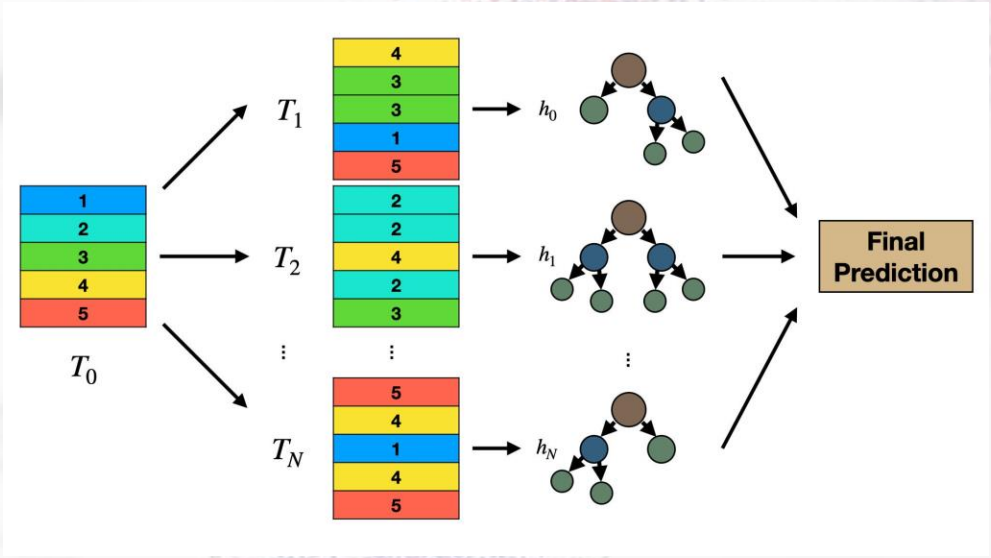
$$k(\mathrm{x}_i, \mathrm{x}_j) = \exp\left(-\frac{\|\mathrm{x}_i - \mathrm{x}_j\|^2}{2\sigma^2}\right)$$

Decision tree and Random forest for Regression

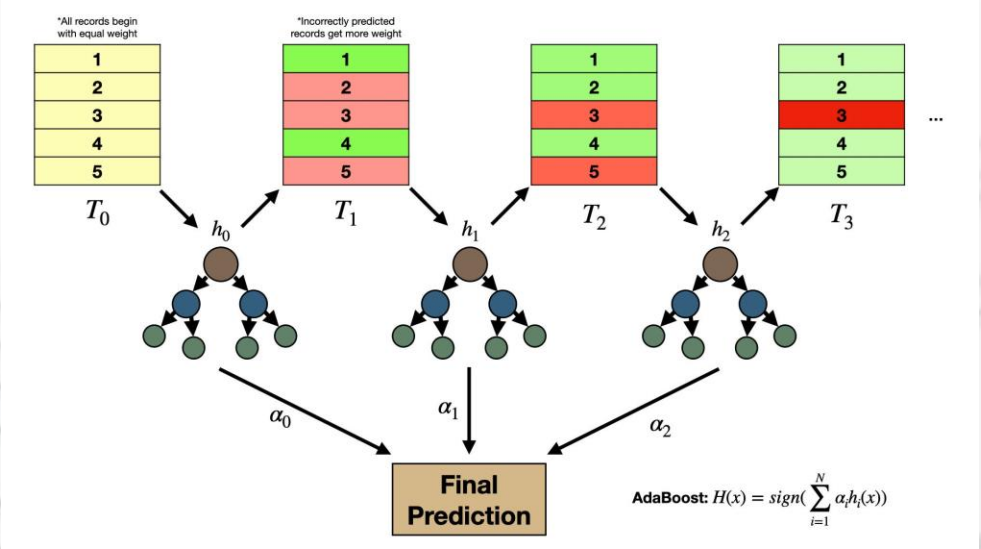


https://towardsdatascience.com/10-decision-trees-are-better-than-1-719406680564/

1) Bagging (Random Forest)



2) Boosting (AdaBoost, Gradient Boosting, XGBoost)



DECISION TREE	RANDOM FOREST
Less accurate result.	More accurate result.
Simple to understand.	Little bit complex to understand.
A decision tree is a decision support tool that uses a tree-like model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility.	Random forests or random decision forests are an ensemble learning method for classification, regression and other tasks that operate by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes or mean prediction of the individual trees.
There is a chance of overfitting.	The risk of overfitting is very minimum.
